Stellenbosch
UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Backseat driving: Using privileged-information auxiliary losses in autonomous racing

by

Jacques Friedrich Amsel

Report submitted in partial fulfillment of the requirements of the module Project (E) 448 for the degree Baccalaureus in Engineering in the Department of Electrical and Electronic Engineering at the University of Stellenbosch

Study leader: Dr. JC Schoeman

October 2026

Acknowledgements

This work would not have been possible without the support of my supervisor, Dr. JC Schoeman, who provided valuable guidance and feedback throughout the project. I would also like to thank my friends and peers for their encouragement and support, as well as my family for their support throughout my studies. Finally, I would like to thank the developers of the open-source software and libraries that were used in this project, including, but not limited to, PyTorch, NumPy, Matplotlib, Seaborn, Pandas, Python, and Ubuntu. Without their contributions, this work would have been immensely more difficult.

"Now, this is podracing!"

- *Anakin Skywalker in Star Wars: The Phantom Menace*

Plagarism declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektueleeiendom van ander persone asof dit jou eie werk is. *Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.*
2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is. *I agree that plagiarism is a punishable offence because it constitutes theft.*
3. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelikse aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is. *Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.*
4. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie. *I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.*

26938839 Student number	 Signature
J.F. Amsel Initials and surname	26 October 2026 Date

Abstract

End-to-end reinforcement learning (RL) agents can drive a car directly from raw sensor inputs, but a LiDAR-only agent must implicitly learning quantities it cannot observe, such as its own velocity, tyre slip, and the curvature of the track ahead to perform well. In simulation, these quantities are available as privileged information, yet standard RL provides no direct signal to learn them; even when the reward is dense, it is still an indirect and high-variance teaching signal for these quantities, so the agent learns them slowly. This project investigates whether auxiliary losses that predict privileged information can be applied to improve the performance and training speed of a LiDAR-only end-to-end racing agent, and whether they change what the agent's internal representations encode.

We find that close-ahead track curvature prediction has a positive effect on the training speed, racing performance, crash rate, and representation quality of the agent, while other losses such as velocity and slip prediction have little-to-no effect.

Table of contents

Acknowledgements **i**

Plagarism declaration **iii**

Abstract **iv**

Notation **viii**

Conventions . . . viii

Reference . . . viii

Acronyms and abbreviations **x**

1. Introduction **1**

1.1. Background and motivation . . . 1

1.2. Problem Statement and Objectives . . . 1

1.3. Solution Overview and Scope . . . 2

1.4. Report Outline . . . 2

2. Background and Related Literature **4**

2.1. Previous Approaches . . . 4

2.1.1. Classical approaches . . . 4

2.1.2. Learning approaches . . . 4

2.2. Learning Methods Using Privileged Information . . . 5

2.2.1. Asymetric Actor-critic Methods . . . 5

2.2.2. Teacher-student distillation . . . 5

2.2.3. Auxiliary learning . . . 5

2.2.3.1. Auxiliary Rewards . . . 6

2.2.3.2. Auxiliary losses . . . 6

2.3. Evaluation of existing approaches . . . 6

3. System overview **8**

4. Simulator **9**

4.1. Simulator performance comparison . . . 10

4.2. Unity Vehicle Model . . . 11

4.2.1. The RigidBody . . . 11

4.2.2. The WheelCollider . . . 11

4.2.3. Steering . . . 13

4.2.4. LiDAR . . . 13

4.3. The track . . . 13
4.3.1. Centerline generation . . . 13
4.3.2. Track walls . . . 15
4.3.3. Track Curvature . . . 16

5. The Agent **17**
5.1. Partially Observable Markov Decision Processes (POMDPs) . . . 17
5.1.1. Value functions . . . 18
5.2. Reinforcement Learning (RL) . . . 19
5.2.1. Policy gradient theorem . . . 19
5.2.2. REINFORCE . . . 20
5.2.3. Proximal Policy Optimisation (PPO) . . . 20
5.3. Neural Network Architecture . . . 21
5.3.1. Action Distributions . . . 22
5.3.2. Auxiliary heads . . . 23
5.3.3. Choice of Auxiliary Loss Function . . . 24
5.4. Finding the Optimal Weights . . . 26
5.4.1. Hyperparameter Tuning . . . 26
5.4.2. Adaptive Weighting . . . 27
5.4.2.1. PCGrad . . . 28
5.5. Metrics . . . 28
5.5.1. Progress . . . 28
5.5.2. Crash rates . . . 28
5.5.3. Reward function . . . 29
5.5.4. Area Under the Curve (AUC) . . . 29
5.6. Trainer Performance Tuning . . . 29

6. Experiments & results **31**
6.1. Baseline Model Performance . . . 31
6.2. Learned features of Baseline Model . . . 31
6.3. Agent Performance After Auxiliary Learning . . . 33
6.4. Learned Features After Auxiliary Learning . . . 35

7. Conclusions and recommendations **36**
7.1. Summary . . . 36
7.2. Future work . . . 36

Bibliography **38**

Appendix **45**
Appendix C Project Timeline . . . 46
Appendix D Graduate Attributes . . . 47
Appendix E Complex Engineering Problem . . . 47
Appendix F AI Declaration . . . 48

Appendix G WheelCollider properties 49
Appendix H Proportion of Data Within 1 Standard Deviation 50
Appendix I Auxiliary Head Regression Results 51
Appendix J Training Performance Benchmark Results 52

Notation

Conventions

The symbols in this thesis follow a small set of conventions, so a new expression can usually be read without looking it up.

- **Capital vs. lowercase.** A capital letter is a random variable (S_t, A_t, O_t), and the matching lowercase letter is a particular realized value of it (s_t, a_t, o_t).
- **Calligraphic letters** denote sets or spaces: $\mathcal{S}$, $\mathcal{O}$, $\mathcal{A}$ are the state, observation, and action spaces. Lowercase elements live in the corresponding space, e.g. $s \in \mathcal{S}$.
- **Subscript t** indexes time, so x_t is the quantity x at time step t.
- **Subscript θ** marks dependence on learnable parameters, e.g. π_θ.
- **Superscript π** means "under policy π", e.g. V^π, Q^π, A^π.
- **Hat** marks an estimate: $\hat{X}$ estimates the random variable X, and $\hat{x}$ is a sample estimate of it.
- **Tilde** means "distributed as": $X \sim Y$ is exact, and $X \dot{\sim} Y$ is approximate.
- **Conditioning bar.** In $f(a \mid b)$, quantities after the bar are given. This holds for both probabilities and expectations.
- **Expectation subscripts** say what the expectation is over: $\mathbb{E}_\pi$ is over trajectories generated by policy π, and $\mathbb{E}_{x \sim p}$ is over x drawn from distribution p.

Reference

Reinforcement learning setup

$\mathcal{S}$	Set of all states
$\mathcal{O}$	Observation space
$\mathcal{A}$	Action space
S_t, s_t	State at time t (random variable, realization)
O_t, o_t	Agent's observation at time t (random variable, realization)
A_t, a_t	Action taken at time t (random variable, realization)

r_t	Reward received at time t
$\rho_0(s)$	Initial state distribution
γ	Discount rate
λ	GAE discount rate

Policies and value functions

π_θ	Policy with parameters θ
$\pi_\theta(a_t \mid o_t)$	Probability density of action a_t given observation o_t under policy parameters θ
$G_t = \sum_{i=t}^{T} \gamma^{i-t} \cdot r_i$	Discounted return at time t
$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$	Value function for policy π at state s
$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$	Action-value function for policy π at state s and action a
$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$	Advantage function for policy π at state s and action a

Probability and statistics

$\Delta_{\mathcal{Y}}(X)$	Set of all probability distributions over variable X with support $\mathcal{Y}$
$X \sim Y$	X is distributed according to Y
$X \mathrel{\dot\sim} Y$	X is approximately distributed according to Y
$\mathbb{E}[X]$	Expected value of X
$\mathbb{E}_{x\sim p}[f(x)]$	Expected value of $f(x)$ when x is distributed according to p
$\mathrm{Var}(X)$	Variance of X
$N(\mu, \sigma^2)$	Gaussian distribution with mean μ and variance σ^2
$\mathrm{Uniform}(a, b)$	Uniform distribution with support $[a, b]$, evaluated at x
$\hat{X}$	Estimator for X
$\hat{x}$	Sample estimate of X

Acronyms and abbreviations

AUC	Area under the curve
C.I	Confidence interval
FOV	Field of view
GAE	Generalised advantage estimation
H2H	Head-to-head
IMU	Inertial measurement unit
LiDAR	Light detection and ranging
MDP	Markov decision process
MAE	Mean absolute error
MSE	Mean squared error
POMDP	Partially observable Markov decision process
PPO	Proximal policy optimisation
RL	Reinforcement learning
RMSE	Root mean squared error

Chapter 1

Introduction

1.1. Background and motivation

Road traffic crashes remain a leading cause of death and serious injury worldwide [1], and the overwhelming majority are attributed to human error [2], as cited by Medina et al. [3]. Autonomous driving promises to reduce severe and fatal accidents [4], improve traffic flow [5], and reduce emissions [6]. Before such systems can be trusted on public roads, however, they must handle difficult situations, where small mistakes compound quickly. Autonomous racing is a demanding proxy for these situations because high speeds, tight control margins, and short recovery times stress the driving policy

Broadly, autonomous driving control architectures fall into two paradigms: modular pipelines and end-to-end learning. Modular pipelines decompose the driving task into distinct sub-problems, such as perception, planning, and control; this provides interpretability and simplifies debugging, but often demands extensive domain expertise and manual tuning. End-to-end learning instead maps raw sensor inputs (e.g., LiDAR or camera frames) directly to control outputs (e.g., speed and steering angle). It avoids hand-engineered intermediate representations, but typically requires vast amounts of data and suffers from poor interpretability, making failures difficult to diagnose [7].

Auxiliary learning, as popularised by Jaderberg et al. [8], presents a compelling method to address these shortcomings. By training a model to perform secondary tasks alongside its primary control objective, the network is encouraged to learn richer, more structured intermediate representations [9] while improving interpretability [7]. In simulation, privileged ground-truth state is available during training even when it is unavailable to the deployed policy. This motivates the question of whether such auxiliary signals can improve an end-to-end racing agent that must drive from limited onboard sensing.

1.2. Problem Statement and Objectives

An end-to-end reinforcement learning (RL) agent that drives from LiDAR alone has to learn representations of privileged information that it cannot directly observe in the real world: its own velocity and slip, upcoming track curvature, and, in Head-to-Head (H2H) racing, the opponent's location. Early in training, the agent has not yet learned

to drive well enough to explore the track effectively, so the reward signal that would teach it these representations is sparse and noisy. This can lead to slow convergence. This can slow convergence and lead to sub-optimal policies. Privileged information is, however, available in simulation.

This report investigates whether *auxiliary heads* that predict privileged information from the policy's shared representation can improve a LiDAR-only end-to-end racing agent. The objectives are to:

(a) select an appropriate simulation environment for autonomous racing;
(b) implement a baseline end-to-end RL agent that drives from LiDAR alone;
(c) quantify what representations the baseline agent natively encodes;
(d) implement auxiliary learning to provide additional supervisory signals to the agent during training; and
(e) compare the auxiliary-learning agent against the baseline on driving performance and representation quality.

1.3. Solution Overview and Scope

The project was executed in three phases: simulation development, baseline evaluation, and auxiliary learning experimentation. First, a custom 3D racing simulator was built in Unity using PhysX-based vehicle dynamic and a procedural track generator. This custom simulator was benchmarked against the standard F1Tenth Gym [10]. Second, a baseline end-to-end reinforcement learning (RL) agent was trainged with Proximal Policy Optimisation (PPO) and a dense progress-based reward. Third, auxiliary heads were added to the network to predict privileged state information during training. The motivation for this approach is developed in the following chapters.

The scope is limited to simulation; no sim-to-real transfer to a physical RoboRacer was attempted. The agent drives from LiDAR and its current steering and throttle only, using a memoryless feedforward policy, and a recurrent policy was also evaluated. Learning is done using PPO, and auxiliary losses are studied; auxiliary rewards and reward shaping are discussed as background but not evaluated. Only single-agent racing is studied, so opponent information is not considered.

1.4. Report Outline

Chapter 2 reviews the background and related literature. It contrasts classical module pipelines with end-to-end learning approaches to autonomous racing, and surveys how rivileged information has previously been exploited through other methods. The chapter closes by identifying the gap in the literature that this report addresses: auxiliary losses have not been studied for LiDAR-only end-to-end racing agents, and it is unknown which privileged targets help or how they reshae the policy's representation.

Chapter 3 provides a brief system overview, describing how the simulator and python training script communicate, and how observations, actions, and rewards flow between them.

Chapter 4 presents the custom Unity simulator. It justifies the choice of simulator, benchmarking it against the F1Tenth gym, and describe the vehicle dynamics implemented, as well as the procedural track generation and privileged information available during training.

Chapter 5 decsribes the agent and auxiliary learning method. It formalises racing as a POMDP, introduces PPO and the network architecture, and defined the auxiliary targets, loss functions, and weighting strategy, together with the metrics used for evaluation.

Chapter 6 reports the expirements performed and their results. It first characterises the baseline agent's performance, and, using linear probes, what its internal representation natively encodes; it then evaluates each auxiliary configuration against the baseline on diving performance and representation quality. This chapter tests the central claim that auxiliary losses improve both representation learning and racing behaviour.

Chapter 7 concludes the report. It summarises the findings against the objectives, discusses limitations, and outlines future work, thereby placing our contributions in the context of the broader autonomous racing literature.

Chapter 2

Background and Related Literature

2.1. Previous Approaches

Evans et al. [11] present a survey breaking down the autonomous racing literature in RoboRacer into two categories: *classical approaches* and *learning approaches*. We shall provide a brief overview of classical approaches, and then focus on learning approaches, which are more relevant to this project.

2.1.1. Classical approaches

Classical approaches generally decompose the driving task into three stages: perception, planning, and control [11]. Perception is the process of converting raw sensor inputs into a representation of the environment, such as a map of the track and the location of other vehicles. Planning uses this representation to generate a trajectory for the vehicle to follow, and control executes this trajectory by sending commands to the vehicle's actuators. These approaches tend to be more interpretable and easier to debug [12], as each component can be analysed separately [13].

2.1.2. Learning approaches

Learning approaches, which have recently shown promise in the field of autonomous driving in simulation [14], [15] as well as in deployment on a physical vehicle [16], are often end-to-end, mapping raw sensor inputs directly to control outputs [13]. This can be done using supervised learning (such as behaviour cloning [17]), where the model is trained on a dataset of expert demonstrations, or using reinforcement learning [18], where the model learns by interacting with the environment and receiving feedback in the form of rewards.

Learning approaches have the potential to learn complex behaviours that are difficult to hand-engineer, and can adapt to changing environments [19]. However, they often require large amounts of data and computational resources [20], and can be less interpretable than classical approaches: deep end-to-end learning approaches are often regarded as "black boxes" [21], which may be undesirable in safety-critical

applications [22]. In RoboRacer, learning approaches have shown promise, but were unable to outperform classical approaches in benchmarks [11], and recently the majority of 27th RoboRacer Autonomous Racing Competition entries were classical approaches [23], despite learning approaches generalising better to unseen tracks [19].

2.2. Learning Methods Using Privileged Information

During training in simulation, information that is not available to the agent at inference time, is available. This information is referred to as *privileged information*, and is usually unused. There are, however, several methods that use privileged information to improve the performance of an agent.

Common ways to use privileged information include as inputs to training-only components (e.g., an asymmetric critic), as inputs to a teacher for distillation, and as auxiliary rewards or losses. We provide these examples to illustrate how other methods use privileged information, and to motivate the use of auxiliary losses as an alternative method in this project.

2.2.1. Asymetric Actor-critic Methods

Asymetric actor-critic methods [24] are a variant of actor-critic methods that train two distinct models: one for the policy, which chooses the agent's actions and only receives the agent's observations as input, and one for the critic, which receives additional privileged informaton as input. The critic is used to estimate the value of each state, which is then used to update the agent to choose actions that lead it into more valuable states. Vasco et al. [25] show that an asymmetric critic can achieve better-than-human performance using a vision-based RL agent in a simulated autonomous racing environment, and perform better than a baseline symmetric actor-critic agent.

2.2.2. Teacher-student distillation

Teacher-student distillation [26] involves training a teacher model with access to privileged information, and then using the teacher's outputs to train a student model that does not have access to this information. The student model is trained to mimic the teacher's behaviour, effectively learning from the teacher's understanding of its privileged knowledge. Xu et al. [27] applied this method to LiDAR-based autonomous racing on the RoboRacer platform, and found that the student model was able to achieve better performance than a baseline model trained without a teacher.

2.2.3. Auxiliary learning

Auxiliary learning is a method that falls under the umbrella of multi-task learning, where a model is trained to perform multiple tasks simultaneously. Specifically, in auxiliary learning, the model's performance on the auxiliary task is not an objective

in itself, but the model's performance on the auxiliary task is used to improve its performance on the primary task. This occurs because the auxiliary task provides additional information that can be used to improve the model's understanding of the primary task, which can lead to better generalisation and faster learning [28].

2.2.3.1. Auxiliary Rewards

When privileged information is available, it can be used to construct auxiliary rewards that guide the agent toward desirable behaviors by evaluating performance against sub-goals. For example, in autonomous racing, privileged knowledge of track layout or opponent positions can generate bonuses for staying centered or avoiding near-misses.

While auxiliary rewards act as a form of reward shaping - supplying denser feedback to accelerate learning - there is a crucial distinction between auxiliary rewards and reward shaping. Traditional potential-based shaping, where the agent is, for example, rewarded for moving towards a target and penalised for moving away from it, guarantees that the underlying task and optimal policy remain unchanged [29]. Auxiliary rewards, however, act as heuristic bonuses that are rarely potential-based. Consequently, they risk altering the task being solved, which can cause the agent to converge on suboptimal policies for the original objective [29].

2.2.3.2. Auxiliary losses

Incorporation of *auxiliary heads* that predict privileged into a model, and using supervised *auxiliary losses* to train them, has emerged as a promising technique for improving the performance, interpretability, and training time of end-to-end learning approaches. It does so in three ways: it reduces the variance of the learning signal, it allows privileged information to be used even when it is unavailable at inference time, and it improves representation quality [8], [30].

Examples of auxiliary learning with privliged information prediction include Lample et al. [31] who demonstrated outstanding performance in first-person shooter games by training an agent to predict the presence of enemies in the agent's field of view (FOV), and Mirowski et al. [32] who found auxiliary tasks for depth prediction and loop closure detection improved navigation in 3D mazes, even compared to agents trained with depth information directly as input.

In the field of autonomous driving, Chitta et al. [33] applied auxiliary losses for depth prediction, semantic segmentation, and vehicle detection, achieving state-of-the-art performance on the CARLA benchmark, and significantly decreasing the crash rate. Chen et al. [7] use auxiliary losses for birds-eye-view prediction in urban environments, improving both interpretability, and performance.

2.3. Evaluation of existing approaches

End-to-end RL agents on the RoboRacer platform generalise well to unseen tracks but have yet to match classical pipelines, and their opaque representations make

failures hard to diagnose. For example, one may not understand whether an agent chose to accelerate into a wall because it misjudged its current velocity, or because it thought the track was straighter than it actually is. In simulation, privileged state such as velocity, tyre slip and upcoming track geometry is available but cannot be used at deployment, when these quantities are unavailable. Existing ways of exploiting priviledged information during training each have a limitation: An asymmetric critic improves value estimation but doesn't directly shape the policy's own representation; teacher-student distillation is complex since it requires a separate teacher and a two-stage pipeline; and auxiliary rewards alter the optimisation objective and must still pass through the high-variance policy gradient. Auxiliary losses avoid all three: they leave the RL objective unchanged, supply a lower-variance supervised gradient directly to the shared representation, and are simple to implement. They have improved performance in first-person games, 3D navigation and urban driving. However, to our knowledge, they have not been studied for LiDAR-only racing, nor has it been examined which privileged targets help, or how they change what the policy's representation encodes. This project addresses that gap.

Chapter 3

System overview

The system trains an end-to-end RL agent to drive a simulated RoboRacer vehicle from as input LiDAR. It runs entirely in simulation and has two subsystems, shown in Figure 3.1: a simulator, which models the vehicle, track and physics interactions, and a Python training script, which computes rewards given the state communicated by the simulator, and updates the policy. The two communicate over gRPC through the ML-Agents toolkit [34], [35].

At each timestep t, the simulator sends a subset of the complete state, $b_t \in s_t$, to the training script. Only the essential information is sent to keep the communication overhead low. The training script then computes the observation o_t using the partial state information b_t and performs a forward pass through the policy using o_t as input, computing the next action a_t. This action is sent to the simulator, and is applied to the vehicle, stepping the environment forward to the next timestep $t + 1$.

To speed up training, multiple instances of the simulator are run in parallel, each with its own set of agent running in separate sandboxed simulation environments.

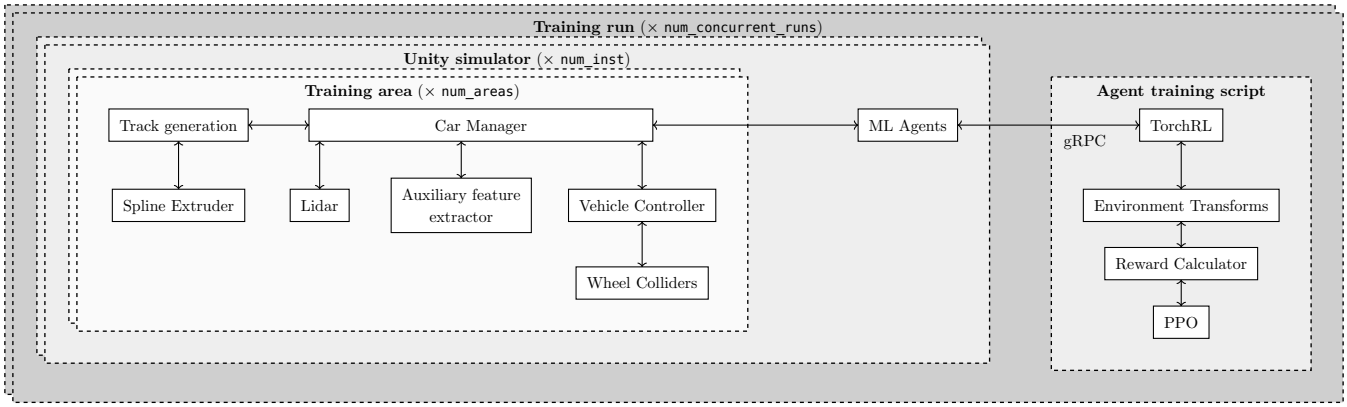


Figure 3.1: System diagram. There are `num_concurrent_runs` concurrent training runs running simultaneously, each with `num_inst` parallel simulator instances. Each simulator instance simulates `num_areas` vehicles, each with its own track. The simulator communicates with the Python training script over gRPC, and the training script updates the policy using PPO.

The following chapters shall outline the design and implementation of the two subsystems: the simulator (Chapter 4) and the training script & agent (Chapter 5).

Chapter 4

Simulator

Previous work on end-to-end driving has largely made use of the F1Tenth gym, with the F1Tenth gym being the most common simulator among methods surveyed in the survey by Evans et al. [11]. The F1Tenth gym is a Python-based 2D simulator for the F1Tenth (now RoboRacer) vehicle. This simulator is easy to use, and runs in Numba-compiled Python, but features a simplified bicycle model of the vehicle, which approximates slip dynamics using a linear function [10].

The AutoDrive simulator [36], which uses the Unity game engine [37], is a 3D simulator that models the vehicle and its environment using PhysX [38], a physics engine developed by NVIDIA. PhysX simulates rigid-body dynamics, including collisions, friction and gravity, and allows for higher-fidelity simulations, including weight transfer dynamics, and suspension deformation in Unity.

Table 4.1: Feature comparison of the Unity (PhysX) and F1Tenth gym simulators

	Unity (PhysX)	**F1Tenth gym**
Integrator	Euler	Runge-Kutta 4th order
Car model	3D, 4-wheel rigid body with suspension, weight transfer, non-linear wheel slip dynamics	2-wheel bicycle model with linearised slip dynamics
Sensors	LiDAR (max 101 rays per agent), IMU, wheel encoders[1], occupancy grids	LiDAR, IMU (excl. pitch, roll)
Track generation	No	In Beta [39]

Despite its higher-accuracy physics, the AutoDrive simulator by default does not provide various features that are important for end-to-end training in our auxiliary learning setup. For example, it is missing track generation & privileged information reporting. Therefore, a custom simulator was implemented in Unity using components from the AutoDrive simulator. Specifically, the `VehicleController` and `IMU` components were not re-implemented, but instead used directly from the AutoDrive simulator. The rest of this chapter describes the custom simulator, including its performance, vehicle model, track generation, and privileged information reporting.

[1]The IMU and wheel encoders are modules available in the AutoDrive simulator, not native Unity

4.1. Simulator performance comparison

In simulation-based training, throughput is critical because agents typically require a very large number of timesteps to learn. There are two kinds of timesteps: at each *control timestep*, the agent receives an observation and outputs an action, and at each *physics timestep*, the physics engine advances the numerical physics integration. The simulation timestep is usually shorter than the control timestep, since the dynamics must be integrated at a finer resolution to remain accurate. Following the settings used by Evans et al. [11], we use a control frequency of 25 Hz and a simulation frequency of 100 Hz.

In order to ensure the custom Unity simulator is suitable for training and not just evaluation, its performance was compared to the F1Tenth gym. A rollout of 10 000 control timesteps (6 minutes and 40 seconds in simulation time) was performed in each environment on an M4 MacBook Air, and the average time per control step was measured. Both environments were run in headless mode to remove any potential overhead from the graphical user interface.

Despite the Unity simulator being significantly more complex than the F1Tenth gym, it runs at a comparable speed, using a comparable amount of CPU resources in this environment, as shown in Figure 4.1, with the Unity simulator being only 19% slower compared to the F1Tenth Gym in terms of mean steps per second (sps).

Due to the relatively minor performance difference, and Unity's more realistic 3D physics, the Unity simulator was chosen for training the RL agent, despite the F1tenth gym being more widely used in the literature.

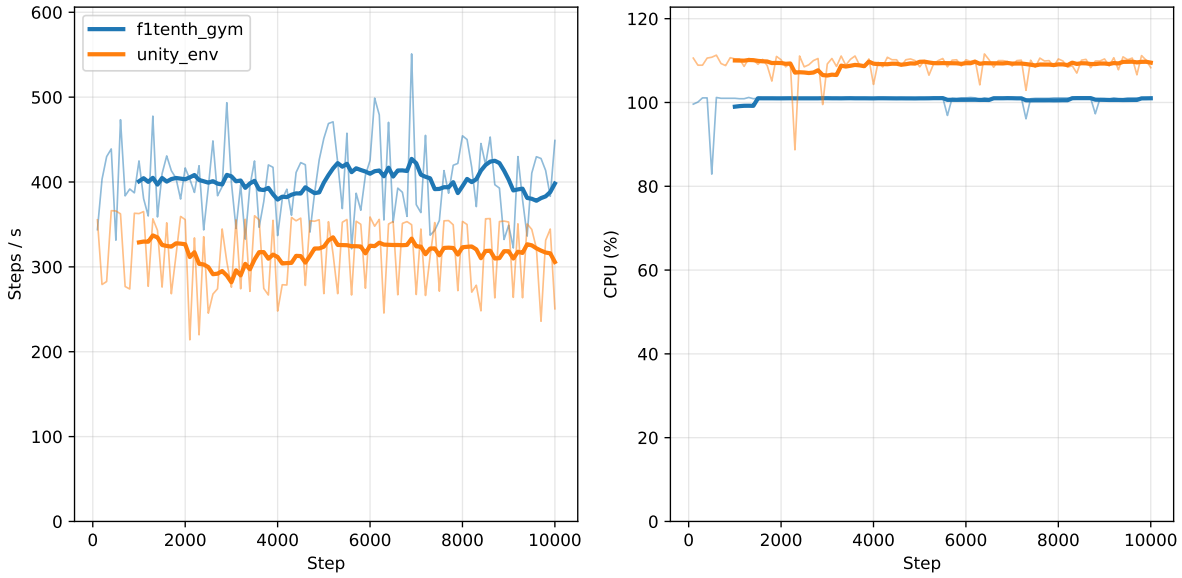


Figure 4.1: Benchmark of Unity simulator vs F1Tenth gym. In the CPU chart, a value greater than 100 indicates that the simulator used more than one CPU core. Includes child processes spawned by the simulator (e.g. in Unity's case)

4.2. Unity Vehicle Model

The vehicle is modelled as a `RigidBody` with four wheels, each simulated by a `WheelCollider`. The two components divide the work: the `Rigidbody` represents the car as a single moving mass, and the `WheelCollider`s generate the forces that move it. Near the front of the car, a `RayPerceptionSensor` from the ML-Agents toolkit simulates a 2D LiDAR sensor with 101 rays, which is used as the agent's observation.

4.2.1. The RigidBody

A `RigidBody` is the Unity component that gives an object rigid physical behaviour. It stores the car's mass, centre of mass and moment of inertia. At each fixed timestep the physics engine (PhysX) integrates the forces and torques acting on it to update its linear and angular velocity and its pose. The car's body is covered by simple box colliders (Figure 4.2), which handle collisions with walls and other objects. They play no part in the tyre-road interaction, which is handled entirely by the wheels. `BoxColliders` were used to model the car's body and collisions between the wheels and walls, since the `WheelCollider` component doesn't detect these kinds of collisions. The mass of the vehicle is set to 3.47 kg based on the values reported by Samak and Samak [40].

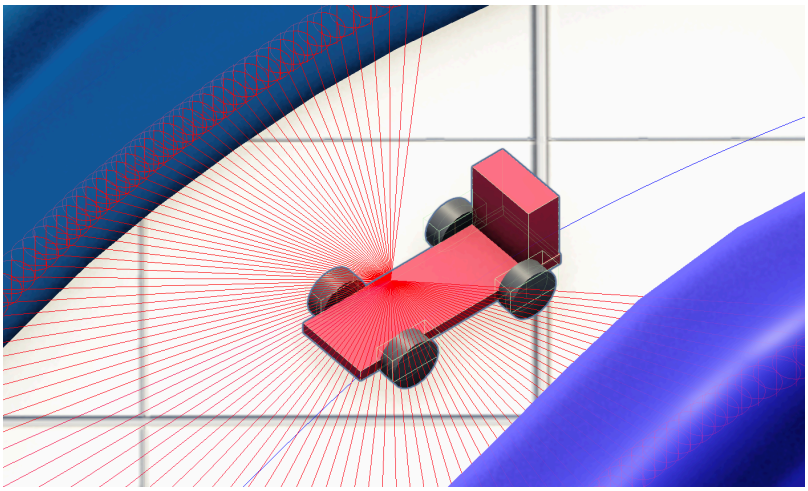
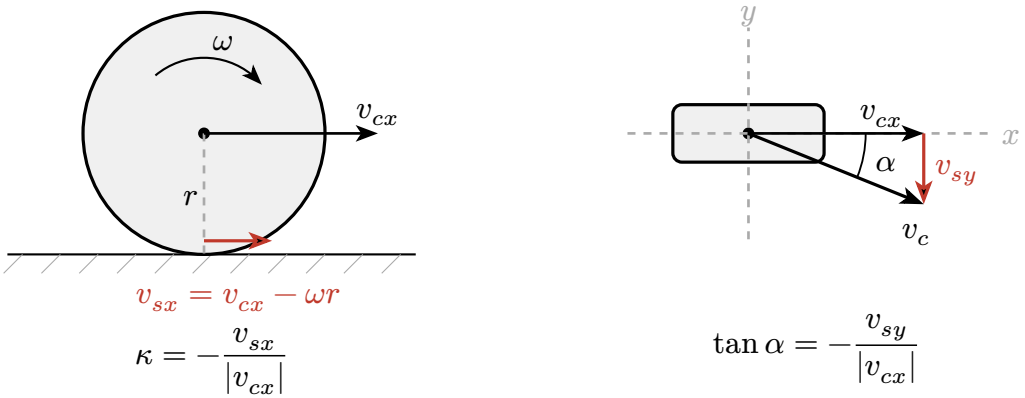


Figure 4.2: Custom Unity car's colliders: the body colliders handle collisions, and the four `WheelCollider`s handle suspension and tyre forces. The red lines show the raycasts by the `RayPerceptionSensor` that simulates the LiDAR. Despite the car looking simpler than the one in the AutoDrive simulator, it has similar physical properties, since we removed the visual meshes and textures.

4.2.2. The WheelCollider

In our custom simulator, each wheel of the RoboRacer is modelled as a `WheelCollider`, which simulates the wheel's suspension, rotation and tyre friction [41]. This kind of collider uses physics similar to that modelled by the semi-empirical "magic formula" wheel dynamics proposed by Pacejka et al. [42], [43].

The tyre model is built on the idea of *slip*. *Longitudinal slip*, κ, is the ratio between the longitudinal slip velocity and the wheel center velocity. *Lateral slip*, $\tan(\alpha)$, is the ratio between the lateral slip velocity and the wheel center velocity. A wheel slips when it spins faster than the wheel center moves (wheelspin), turns slower than the wheel center moves (braking), or is dragged sideways (cornering or drifting). A slip value of 0 therefore indicates a freely rolling wheel, while a slip value of > 1 indicates that the point of contact has a higher velocity than the wheel center.



(a) Side view: longitudinal slip **(b)** Top view: lateral slip

Figure 4.3: Slip quantities. Longitudinal slip compares the slip velocity at the contact point with the wheel-centre velocity; lateral slip compares the sideways slip velocity with the forward velocity.

A tyre needs some slip to produce force. The `WheelCollider` computes the force from the slip using a friction curve (Figure 4.4). Force rises with slip until it reaches a peak, the *extremum*, which is the tyre's maximum grip. Beyond the peak, the tyre begins to slide and the force falls to a lower, constant value, the *asymptote*. Pushing the car harder therefore gives less force, not more, which is a regime a racing agent must learn to avoid. The curve is defined separately for the forward direction (accelerating and braking) and the sideways direction (cornering). Note that in real life, at low slip values, the rubber stretches instead of actually sliding on the road. This is modelled by the rising part of the curve at low slip values.

Figure 4.4 shows the difference between a Unity-style piecewise friction curve and a Pacejka "magic formula" curve that was fit using least-squares to approximate Unity's friction curve. The values used for our simulations are detailed in Section Appendix G and are the values calibrated by Samak and Samak [40] for the RoboRacer platform, ported to our custom Unity simulator.

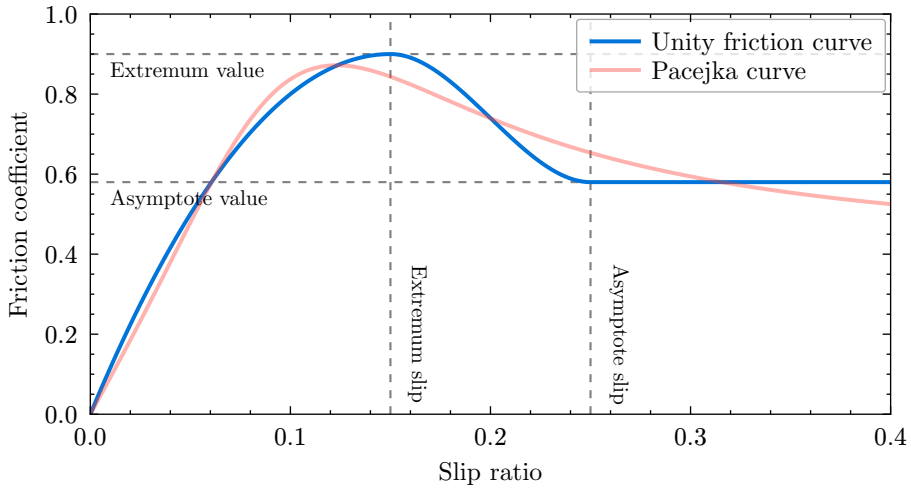


Figure 4.4: Approximated WheelCollider friction curves [41] for the RoboRacer compared to the Pacejka "magic formula" [42]

4.2.3. Steering

On the real-life RoboRacer, steering actuation cannot occur instantaneously, but is limited by the steering servo's maximum angular velocity. The `WheelCollider` component does not model this limitation, so we use the `VehicleController` from AutoDrive to implement a cap on steering speed of $\omega_{\max} = 315°/\,\text{sec}$. This value was also chosen based on the calibration of the RoboRacer platform by Samak and Samak [40].

4.2.4. LiDAR

The vehicle is equipped with a simulated 2D LiDAR sensor, which is implemented using a `RayPerceptionSensor` from the ML-Agents toolkit. The sensor emits 101 rays in a fan pattern, covering a field of view of 270°. Each ray measures the distance to the nearest obstacle in its cast direction, and returns a normalized value between 0 and 1, where 1 indicates no obstacle within the maximum range of the sensor (20m).

4.3. The track

During training, a variety of tracks are used to prevent the policy from overfitting to any single layout. Training on diverse environments has been shown to improve generalisation to unseen environments [44], and is closely related to domain randomisation, where simulator parameters are randomised to produce more robust policies [45], [46].

4.3.1. Centerline generation

To this end, we implemented a procedural track generator in C# in Unity. The idea is a *mean-reverting random walk*: a polyline of N equal-length segments whose heading wanders randomly but is continually pulled back towards the heading of a *reference circle* of radius R. The result wanders freely but still forms a single closed loop of roughly predictable size.

The walk keeps track of two angles: the *reference heading* φ_i, the tangent direction of the reference circle, and a *heading offset* δ_i relative to it. The segment direction is $\theta_i = \varphi_i + \delta_i$. Only the offset is random; the reference heading advances deterministically by $2\pi/N$ per step.

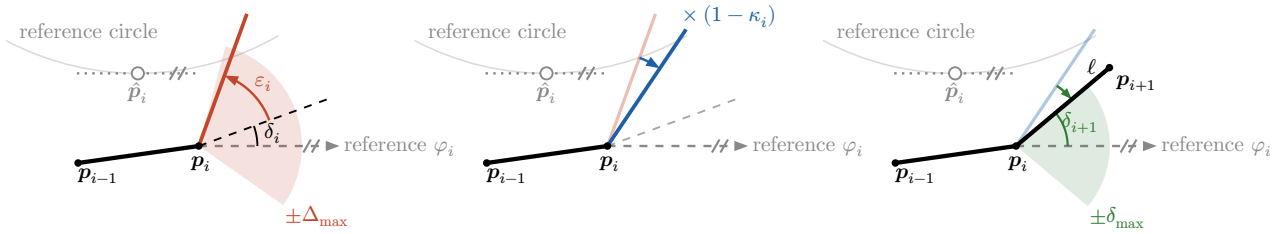
The walk starts at $p_0 = (R, 0)$ with $\delta_0 = 0$, i.e. tangent to the circle. Each segment has length $\ell = 2\pi R/N$, the circumference divided by the number of segments. For $i = 0, 1, ..., N-1$, with progress $t_i = (i+1)/N$:

1. We calculate the reference heading, $\varphi_i = 2\pi t_i + \pi/2$, the tangent of the reference circle at $\hat{p}_i = R \cdot (\cos 2\pi t_i, \sin 2\pi t_i)$.
2. We take a random turn (Figure 4.5 (a)), drawing $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ and adding it to the offset.
3. As depicted in Figure 4.5 (b), we shrink the offset towards zero by a factor $1 - \kappa_i$, where

   $$\kappa_i = \kappa_0 + (1 - \kappa_0) t_i^s.$$

   The restoring strength κ_i starts at κ_0 and ramps up to exactly 1 on the final step. The exponent s controls how late the ramp kicks in.
4. In Figure 4.5 (c), we limit the offset from the reference heading to prevent accumulation of large offsets, and advance by length l:

   $$\delta_{i+1} = \mathrm{clip}((1 - \kappa_i)(\delta_i + \varepsilon_i), -\delta_{\max}, \delta_{\max}),$$
   $$p_{i+1} = p_i + \ell(\cos(\varphi_i + \delta_{i+1}), \sin(\varphi_i + \delta_{i+1})).$$



(a) Random turn (b) Mean reversion (c) Clip and step

Figure 4.5: One step of the track-generation walk. **(a)** A random turn $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ is added to the carried offset δ_i, measured from the reference heading φ_i. **(b)** A fraction κ_i of the accumulated offset is removed, pulling the heading towards the reference φ_i. **(c)** The offset is clipped to $\pm\delta_{\max}$, giving δ_{i+1}, and a step of length ℓ is taken to $\boldsymbol{p}_{i+1}$.

Early on, $\kappa_i \approx \kappa_0$ is small, so the offset behaves like a weakly damped random walk (an AR(1) process) and the track meanders. Near the end, $\kappa_i \to 1$ forces the heading back onto the reference tangent, so the walk heads towards the start point. Because the integrated heading still deviates slightly from the circle, p_N generally misses p_0 as visualised in Figure 4.6 (a). We close it by distributing the gap linearly along the track:

$$p_i \leftarrow p_i + \frac{i}{N}(p_0 - p_N), \quad i = 0, ..., N.$$

The gap is (usually) small, so this correction barely distorts the track. Finally, we fit a Catmull-Rom spline to the polyline to smooth out the corners, as visualised in Figure 4.6 (c).

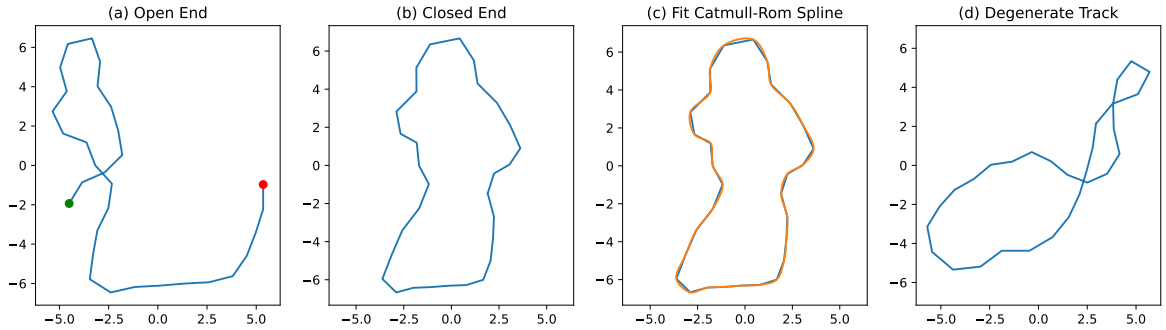


Figure 4.6: (a) Open end of the track before distributing the gap between p_0 (green) and p_N (red) **(b)** Closed track after distributing the gap **(c)** Catmull-Rom spline interpolation of the polyline **(d)** A degenerate track

This method is not immune to generating self-intersecting (degenerate) tracks, as shown in Figure 4.6 (d), but the probability is low enough that we simply discard any degenerate tracks and generate a new one. Empirically we find 50 tracks of 100 000 to be degenerate with our chosen parameters, so a 95%-confidence interval (C.I.) for $P(\text{degenerate track} \mid N = 30, \Delta_{\max} = 80°, \kappa_0 = 0.1, s = 3, \delta_{\max} = 90°)$ is $(0.037\%, 0.065\%)$ assuming a binomial distribution.

4.3.2. Track walls

Adding track walls to the generated centerline spline is a non-trivial problem. A naive approach is to scale the centerline's vertices outwards by a fixed multiple, or provide a fixed offset from the center, but this can create undrivably narrow sections, as visualised in Figure 4.7 (a) and (b) respectively. We therefore generate the walls using the Clipper2 library [47], which implements a polygon offsetting algorithm inspired by Vatti [48]. The Clipper2 algorithm is robust to self-intersections, but may produce inset "islands" for large inset distances, or for tracks which are close to self-intersecting, as visualised in Figure 4.7 (c).

Empirically, we find that 1350 of the 100 000 tracks produce these islands (C.I. $(1.28\%, 1.42\%)$), so we simply discard any tracks with islands and re-generate them.

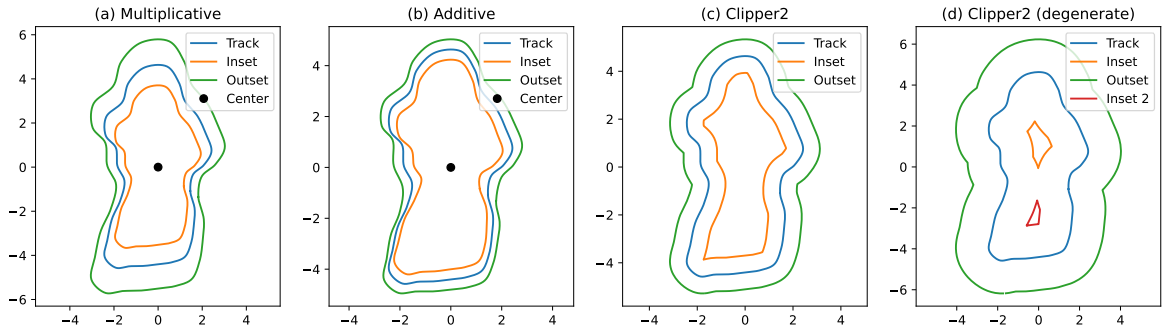


Figure 4.7: Track insets and outsets using different methods

4.3.3. Track Curvature

During training, we wish to observe the effect of using track curvature as an auxiliary loss. To this end, we compute the curvature of the track at 16 equally-spaced points along the track ($d \in \{0.5, ..., 8\}$ m) ahead of the car's current position. The curvature is computed using the formula $c_d = \frac{1}{r_d}$, where r_d is the radius of the osculating circle at that point on the track. The osculating circle was introduced by Leibniz [49] as cited by [50, p. 419, n. 6]; it is the circle that best approximates a curve at a given point, and its radius is computed from the first and second derivatives of the track's parametric equations [51, p. 910-911]:

$$\frac{1}{\kappa} = r = \frac{\left(x'(t)^2 + y'(t)^2\right)^{\frac{3}{2}}}{|x'(t)y''(t) - y'(t)x''(t)|} = \frac{|\boldsymbol{r}'(t)|^3}{|\boldsymbol{r}'(t) \times \boldsymbol{r}''(t)|}$$

Chapter 5

The Agent

In this chapter, we describe the architecture of the agent and the auxiliary losses used to improve its performance. Because we are comparing the effect of auxiliary losses, we train multiple agents with the same architecture, but with different auxiliary losses enabled each time.

We shall first formalise the problem of autonomous racing as a partially observable Markov decision process (POMDP) in Section 5.1, then describe the process of reinforcement learning in Section 5.2, describe the architecture of the agent in Section 5.3, and discuss how the weights of the auxiliary losses are chosen in Section 5.4. Finally, Section 5.5 describes the metrics used to evaluate the performance of the agent.

5.1. Partially Observable Markov Decision Processes (POMDPs)

In order to formally describe the problem of autonomous racing, we first introduce the framework of partially observable Markov decision processes (POMDPs). POMDPs are a generalisation of Markov decision processes (MDPs) that account for the fact that an agent may not have access to the full state of the environment. In a POMDP, the agent receives observations that provide partial information about the underlying environment state, and must make decisions based on this incomplete information.

A POMDPs is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \rho_0, \mathcal{R}, \gamma, \mathcal{Z})$, based on the work by Kaelbling et al. [52]:

At each time step t, the environment is in state $s_t \in \mathcal{S}$, where $\mathcal{S}$ is the state space. This state is a *complete* description of the environment at that time. The first state s_0 is sampled from an initial state distribution ρ_0. The agent does not see s_t directly. Instead, it receives an observation $o_t \in \mathcal{O}$, typically its sensor inputs (e.g. LiDAR, camera frames), drawn from the observation function, $o_t \sim \mathcal{Z}(\cdot \mid s_t)$: the observation function is a stochastic mapping from the true state to the observation, deciding what the agent observes given the true state. The information in s_t that is missing from o_t is called *privileged information*. For example, the car may observe its lidar measurements, but not the road curvature ahead directly, which is privileged information.

Based on its observation, the agent uses its policy π to choose an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. The policy therefore the history of observations to a

probability distribution over actions; we write[2] $a_t \sim \pi(\cdot \mid o_t)$. The policy is what we want to find: it describes the agent's behaviour. The policy depends on the history of observations: we would write $\pi(\cdot \mid o_t, o_{t-1}, ...)$. After the agent has taken action a_t, the environment moves to a new state, $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)$, where $\mathcal{P}$ is the transition function. By the *Markov property*, the next state s_{t+1} depends only on the current state s_t and action a_t, not on earlier history [18]. Then, the agent receives a scalar reward $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$, where $\mathcal{R}$ is the reward function, together with the next observation o_{t+1}. This interaction loop is illustrated in Figure 5.1.

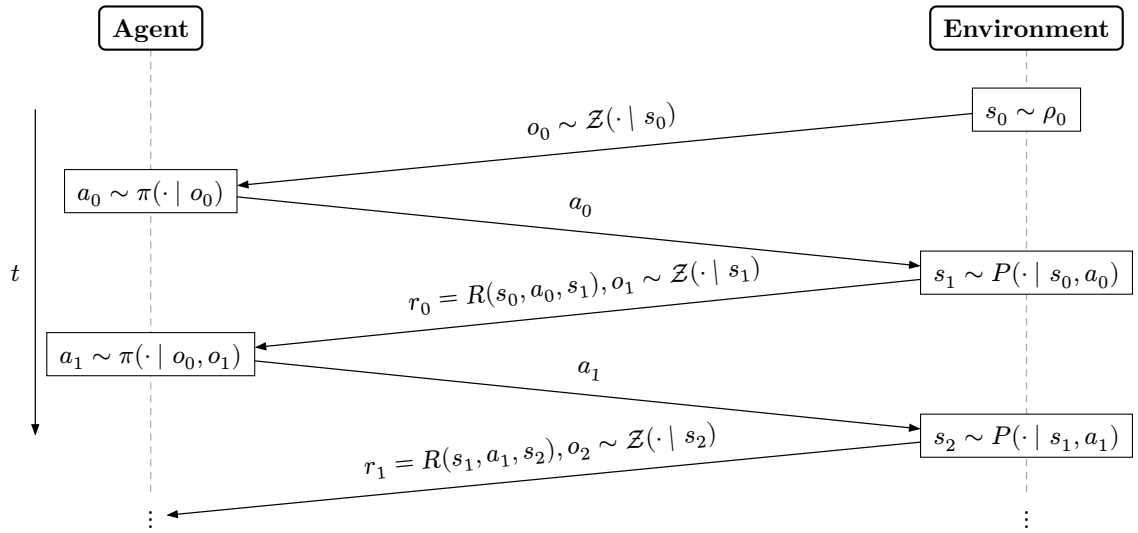


Figure 5.1: Interaction between the agent and the environment over time (t).

The agent's goal is to maximise the *expected discounted return*. The *discounted return* (or simply *return*) from time step t is the discounted sum of all rewards the agent receives from time t onward,

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k},$$

where T is the final time step and $\gamma \in [0, 1]$ is the discount factor. A reward received k steps ahead is scaled by γ^k, so later rewards are worth less than immediate ones: when γ is close to 1 the agent is far-sighted, and when γ is close to 0 it is short-sighted. Since G_t depends on random states, observations and actions, it is a random variable, and the agent maximises its expectation, $\mathbb{E}[G_0]$, from the start of the episode. The policy that achieves this maximum is the *optimal policy* π^*.

5.1.1. Value functions

Value functions measure how good it is for the agent to be in a state, or to take an action in that state, and are the basis for the critic in PPO. The state-value function,

[2]Some formulations may define the policy as a mapping from the true state estimate to actions instead, but this is not an end-to-end approach, since there must exist a state estimator $e : \mathcal{O} \to \mathcal{S}$ in that case: $\hat{s}_t = e(o_t), a_t \sim \pi(\cdot \mid \hat{s}_t)$

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

is the expected discounted return starting in state s and then following policy π thereafter. The action-value function,

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

measures the expected discounted return starting in state s, taking action a, and then following policy π thereafter. Their difference is the advantage function,

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s), \tag{1}$$

which measures how much better action a is than the policy's average action in state s, following policy π.

5.2. Reinforcement Learning (RL)

In order to find the optimal policy, π^* that maximises the expected discounted return, $J(\theta) = \mathbb{E}_\pi[G_0]$, we use reinforcement learning (RL) as introduced by Sutton and Barto [18]. In RL, the agent learns to maximise its expected discounted return by interacting with the environment and receiving feedback in the form of rewards. The agent updates its policy based on the observed rewards and transitions, gradually improving its performance over time. We use RL instead of supervised learning due to lack of labelled data for the optimal policy.

So-called model-free RL algorithms fall into three broad classes: *Value-based* methods learn the optimal action-value function $Q^*(s,a) = \max_\pi Q^\pi(s,a)$ and act greedily with respect to it, choosing the action with the highest value in the current state: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^\pi(s,a)$. *Policy-based* methods, on the other hand, learn a parameterised policy π_θ directly by ascending the estimated gradient of $J(\theta)$ with respect to θ, and do not need the Markov property of the observations or an $\arg\max$ over actions. They can also represent stochastic policies, which may be necessary for optimal behaviour in some POMDPs [53] (although there always exists a deterministic optimal policy in fully-observed MDPs). *Actor-critic* [18] methods combine both: a learned value function (the critic) is used to reduce the variance of the policy gradient, and since the critic is only used in training, it may take privileged information. We focus on policy-based and actor-critic methods in this report.

5.2.1. Policy gradient theorem

Policy gradient methods are a class of RL training algorithms that learn a policy π_θ with parameters θ directly by maximising the expected return $\mathbb{E}[G_0]$ with respect to the policy parameters θ. Given a trajectory $\tau = (s_0, o_0, a_0, r_0, s_1, o_1, a_1, r_1, ..., s_{T-1}, o_{T-1}, a_{T-1}, r_{T-1}) \sim p_\theta$, where p_θ is the probability of the trajectory when following the policy π_θ, we intend to maximise the expected return with respect to the policy parameters θ:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau$$

Taking the gradient of $J(\theta)$ with respect to θ and applying the log-derivative trick [54], we obtain the policy gradient theorem [18]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t\right]. \tag{2}$$

Equation 2 shows in what direction to update the policy parameters in order to increase the expected return. Of course, we cannot compute the expectation exactly without simulating all possible trajectories, and instead have to rely on estimates such as those detailed in Section 5.2.2 and Section 5.2.3.

5.2.2. REINFORCE

The REINFORCE algorithm for policy learning was introduced by Williams in 1992 in the journal *Machine Learning* [54]. It is one of the simplest policy gradient methods: by sampling N trajectories $\tau^{(i)} \sim \pi_\theta$ of length T_i timesteps each, the policy gradient can be estimated in a Monte-Carlo fashion:

$$\widehat{\nabla_\theta J}(\theta) = \frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)} \mid o_t^{(i)}\right) \cdot G_t^{(i)}$$

Although REINFORCE is simple to implement, it has several drawbacks, including having high variance in its gradient estimates, which can lead to slow convergence and instability during training [55], and, being an on-policy algorithm, it can only learn from trajectories generated by the current policy, which can limit exploration and learning efficiency (since it cannot train multiple epochs on the same rollout).

5.2.3. Proximal Policy Optimisation (PPO)

PPO is a modern actor-critic algorithm for policy learning introduced in 2017 by researchers at OpenAI [56]. It improves on REINFORCE in two ways. First, being an actor-critic method, it learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. Second, it optimises a *clipped surrogate objective*, $\mathcal{L}_{\text{PPO}}$, that limits how far each update can move the policy. This prevents large, destructive steps and makes several epochs of updates on the same rollout possible.

Recall that the advantage function in Equation 1 is the difference between the action-value and state-value functions. It therefore measures how much better (positive) or worse (negative) an action is than the policy's average action in that state. In training, $\hat{A}_t = \hat{A}^\pi(s_t, a_t)$ is estimated from observed rewards and the learned value function, $\hat{V}^\pi(s)$, for example with Generalised Advantage Estimation [57]. In PPO, the clipped surrogate objective is given by

$$\mathcal{L}_{\text{actor}} = -\frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} \min\Big(\rho_t(\theta) \cdot \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t\Big)$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}$ is the ratio of the new policy's probability of taking action a_t given observation o_t to the old policy's, and ε is a hyperparameter that controls how much the policy is allowed to change in a single update. The clip function ensures that a policy update that would make $r_t(\theta)$ fall outside of the interval $[1-\varepsilon, 1+\varepsilon]$ is clipped to the nearest endpoint of that interval. This prevents large policy updates that could destabilise training. The min function on the other hand ensures that, while the policy's update is capped when an update makes a good action ($A_t > 0$) more likely, it is not protected from penalties if an update makes a bad action ($A_t < 0$) more likely. These two mechanisms together ensure that the policy is updated in a way that improves performance while avoiding large, destabilising changes that are common in other policy gradient methods [56].

Since PPO is an actor-critic method, it also learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. The value function is learned by minimising the mean squared error between the predicted value and the observed return. The full PPO loss function is therefore a combination of the clipped surrogate objective and the value function loss, along with an entropy bonus to encourage exploration. The final loss function is given by

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}_{\text{actor}} + w_{\text{value}} \cdot \mathcal{L}_{\text{value}} - w_{\text{entropy}} \cdot \mathcal{H}[\pi_\theta]$$

5.3. Neural Network Architecture

The agent is a feedforward actor-critic network implemented in PyTorch [58], with the TorchRL library [59].

It has no recurrent state, so the stack of past scans is its only source of temporal information, since during testing, a Recurrent Neural Network (RNN) performed poorly in comparison. It consists of a shared *trunk* followed by three independent heads, each of which takes as input the same 64-dimensional feature vector output from the trunk, as visualised in Figure 5.2:

- the *policy head* outputs the parameters of the action distribution,
- the *value head* outputs the state-value estimate $\hat{V}^\pi(s)$, and
- the *auxiliary head* outputs predictions of the privileged targets

The trunk (Figure 5.2) has a convolutional part followed by a dense part. The input is a tensor of shape 50×101 (time steps by rays). Three 1D convolutions with ReLU activations slide along the ray axis and treat the time steps as input channels, so the first layer mixes the whole history at each angular position. They have 32, 16, and 16 output channels respectively; all have the same kernel size of 8 (roughly 21° of

FOV). The output is flattened and concatenated with the current steering angle and throttle, and passed through fully connected layers of 128, 72 and 64 units with ReLU activations, followed by layer normalisation of the 64-dimensional output to prevent gradient explosion when the car is under heavy acceleration, pointing the LiDAR above the track walls and resulting in large normalised ray lengths as input (≈ 1).

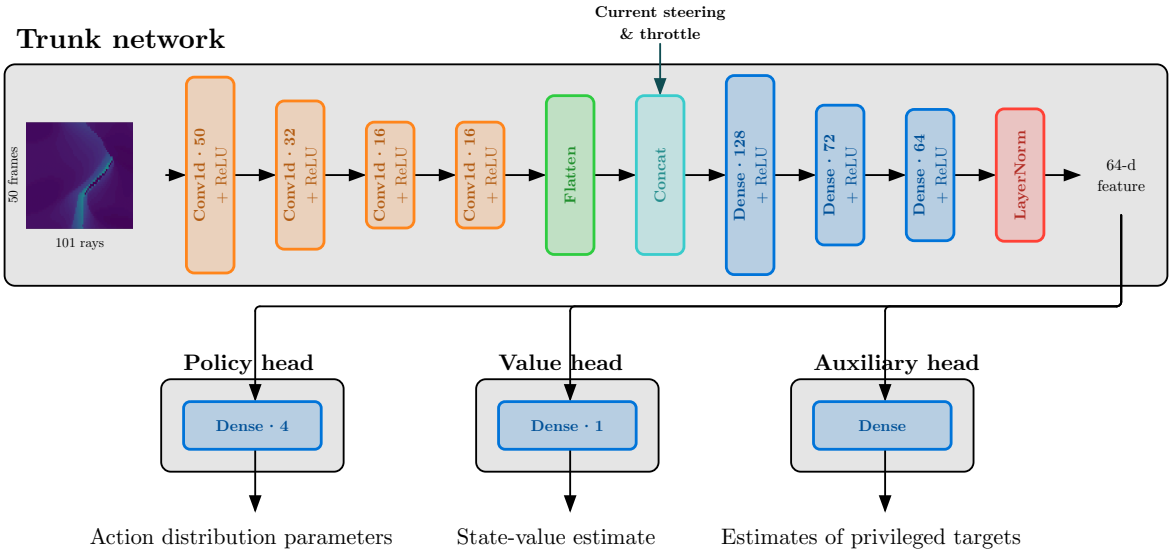


Figure 5.2: The agent's neural network. A 50-frame LiDAR stack passes through four 1D convolutions; the flattened output is concatenated with the current steering and throttle and passed through three dense layers and a layer norm to give a 64-dimensional feature vector. This shared feature vector is then passed to three independent heads: the policy head outputs the parameters of the action distribution, the value head outputs the state-value estimate, and the auxiliary head outputs predictions of privileged targets.

5.3.1. Action Distributions

The policy head outputs a location μ and a raw scale value x for each of throttle and steering. Each action is sampled independently from a tanh-squashed Gaussian,

$$a = \tanh(u), \quad u \sim \mathcal{N}\left(5 * \tanh\left(\frac{\mu}{5}\right), \sigma^2\right),$$

so that $a \in (-1, 1)$, where 1 corresponds to full throttle or full right steering, and -1 corresponds to full reverse or full left steering.

The mean is transformed with a tanh to soft-clamp it to the range $(-5, 5)$ using `tanh_loc=True` in the TorchRL `TanhNormal` distribution. This prevents the mean from growing too large, which would push samples into the saturated region of the squashing function and make it difficult for the policy to recover since gradients vanish there. The constant factor of 5 is the heuristically chosen default factor configured in TorchRL [59].

The standard deviation is made positive with a softplus, with a small floor to ensure that the distribution is never degenerate: $\sigma = \text{softplus}(x) + 10^{-3}$.

The policy head's linear layer uses a weight initialisation with gain 0.01 and zero bias, so every output is close to 0 at the start. Hence $\mu \approx 0$, and both actions are centred near the middle of their range.

This is important because the derivative of the squashing tanh function is

$$\frac{\mathrm{d}a}{\mathrm{d}u} = 1 - \tanh^2(u),$$

which goes to 0 as $|u|$ grows. If the head's outputs start large, samples land in the saturated region, the gradient of the policy loss with respect to μ is almost zero, and the policy cannot recover. In early experiments this is exactly what happened: the car locked at full throttle and full steering in either the positive or negative direction.

5.3.2. Auxiliary heads

Using privileged information during training has the potential to improve the agent's performance, as discussed in Section 2.2. We opt to use privileged information in the form of auxiliary losses, which are additional loss terms that encourage the agent to learn useful representations of the environment. These auxiliary losses are computed using privileged information that is not available to the agent during testing, but can be used during training to improve the agent's performance.

Each frame, the environment emits a set of privileged quantities to the trainer, which are used as *auxiliary targets*. The *auxiliary head(s)* predict these targets during the training process. The auxiliary targets we consider are listed in Table 5.1. During training, the auxiliary losses are added to the PPO loss so that their gradients are combined and backpropagated through the network graph:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{actor}} + w_{\text{value}}\mathcal{L}_V - w_{\text{entropy}}H[\pi_\theta]}_{\mathcal{L}_{\text{PPO}}} + \sum_{k=1}^{K} w^{(k)}\mathcal{L}_{\text{aux}}^{(k)},$$

so that

$$\nabla_\theta \mathcal{L}_{\text{total}} = \nabla_\theta \mathcal{L}_{\text{PPO}} + \sum_{k=1}^{K} w^{(k)} \nabla_\theta \mathcal{L}_{\text{aux}}^{(k)}$$

Table 5.1: The auxiliary targets to be predicted by the agent[3]

Target	Units	Description
Forward velocity v_f	m/s	The component of the agent's velocity in the direction of its forward axis
Lateral velocity v_l	m/s	The component of the agent's velocity in the direction of its lateral axis
Yaw rate ω_y	rad/s	The rate of change of the agent's heading about its vertical axis
Track curvature ahead c_d	1/m	The curvature of the track centerline at d meters ahead of the agent, measured as the reciprocal of the radius of curvature
Per-wheel longitudinal slip κ_x	unitless	The longitudinal slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$
Per-wheel lateral slip $\tan(\alpha_x)$	unitless	The lateral slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$

5.3.3. Choice of Auxiliary Loss Function

The choice of auxiliary loss functions $\mathcal{L}_{\text{aux}}^{(k)}$ has the potential to significantly impact the agent's performance. Since we are predicting continuous quantities, it is natural to use a regression loss function such as Squared Error (SE), Absolute Error (AE), or Huber loss [60], defined respectively as

$$\mathcal{L}_{\text{SE}} = (\hat{y} - y)^2, \quad \mathcal{L}_{\text{AE}} = |\hat{y} - y|, \quad \mathcal{L}_{\text{Huber}}^{\beta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \beta \\ \beta\ |y - \hat{y}| - \frac{1}{2}\beta^2 \text{ otherwise.} \end{cases}$$

Throughout, we use PyTorch's rescaled implementation of the Huber loss (`smooth_l1_loss`), $\frac{\mathcal{L}_{\text{Huber}}^{\beta}(y,\hat{y})}{\beta}$. Dividing by β decouples the threshold from the gradient: the rescaled loss is quadratic with gradient

The Huber loss is a robust alternative to the SE loss that is far less sensitive to outliers. It behaves quadratically for small errors and linearly for large ones, with the transition point set by a threshold parameter β. As visualised in Figure 5.3, this means the gradient magnitude is bounded no matter how large the prediction error grows, whereas the L2 gradient grows without limit. Unlike the L1 loss, whose gradient changes sign abruptly at zero, the Huber loss is smooth there, so the gradient shrinks gradually as the error approaches zero.

[3]In order to ensure the targets are on a similar scale, the targets are normalised to $\sigma = 1, \mu = 0$ using a rollout generated from a pure PPO policy (i.e. without auxiliary losses/heads) as data to estimate σ and μ.

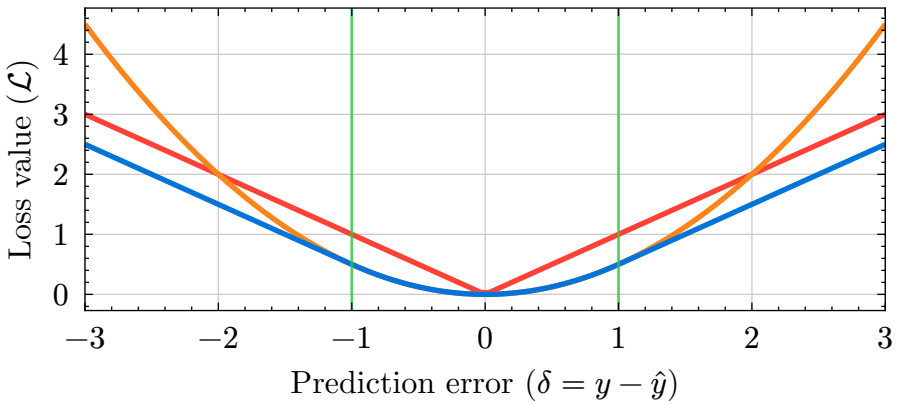


Figure 5.3: Rescaled Huber loss (blue) vs squared error loss (orange) vs absolute error loss (red) for prediction errors δ. Between the green lines, the Huber loss acts like the squared error loss, and outside, its gradient is equal to the gradient of the absolute error loss, so it is less sensitive to outliers than the squared error loss. The Huber loss is smooth at zero, unlike the absolute error loss, so its gradient shrinks gradually as the error approaches zero.

The Huber loss's bounded gradient is particularly useful when the target values have a heavy-tailed, non-normal distribution, since the loss is then far less sensitive to extreme values than the SE loss [61]. This is the case for our auxiliary targets, as visualised in Figure 5.4.

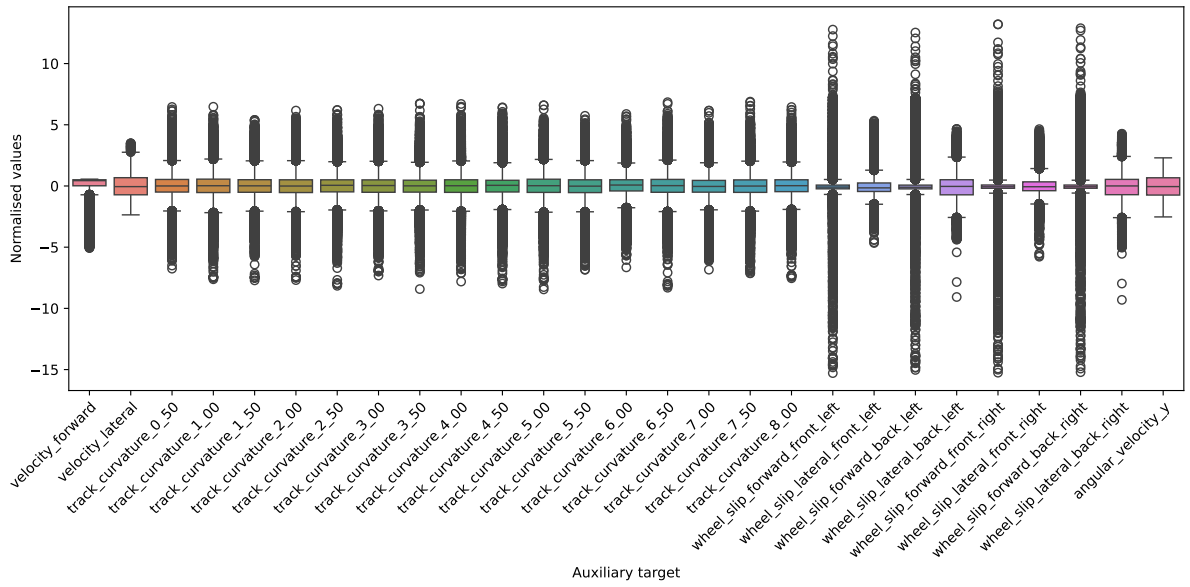


Figure 5.4: Boxplot of the normalised true auxiliary target values collected during a rollout of the baseline policy. It is clear that the targets have so-called "heavy tails".

For this reason, we use the Huber loss for all auxiliary targets. With $\beta = 1$, since the targets are normalised to zero mean and unit variance (Table 5.1), this makes the loss quadratic for errors within one standard deviation - where most[4] of the fitting takes

[4]Proportions of the data within 1 standard deviation of the mean are listed in Section Appendix H

place - and linear, with a gradient of magnitude 1, beyond it, so that the few extreme target values cannot dominate the gradient.

5.4. Finding the Optimal Weights

The question naturally arises: how should the weights $w^{(k)}$ of the auxiliary losses be set? If they are too small, the auxiliary losses will have little effect on the agent's learning. If they are too large, the agent may use all its capacity to optimise the auxiliary tasks and fail to learn a good policy, or the auxiliary task gradients may conflict with the main task gradients, slowing learning. This is an optimisation problem, where the weights are tuned to achieve the best overall performance. Two options exist: choosing the weights before training (hyperparameter tuning), or adapting the weights during training (adaptive weighting).

5.4.1. Hyperparameter Tuning

Hyperparameter tuning is a common approach to finding the optimal weights for auxiliary losses. It involves running multiple training experiments with different weight configurations and then evaluating the agent's performance, choosing the configuration that yields the best results. This approach is simple to implement, but it can be computationally expensive and time-consuming. It also lacks a guarantee of finding the optimal weights, as the search space can be large and complex.

The simplest tuning method is a grid search, where a discrete set of weight values is defined for each auxiliary loss and all possible combinations are evaluated. However, this approach can be inefficient, especially when the number of auxiliary losses is large, as the number of combinations grows exponentially with the number of weights considered.

Random search [62] is another hyperparameter tuning method, where weight configurations are sampled randomly from a predefined distribution. This approach can be more efficient than grid search, as it allows for exploration of a larger search space without evaluating every possible combination. However, it still requires a significant number of training runs to find the optimal weights.

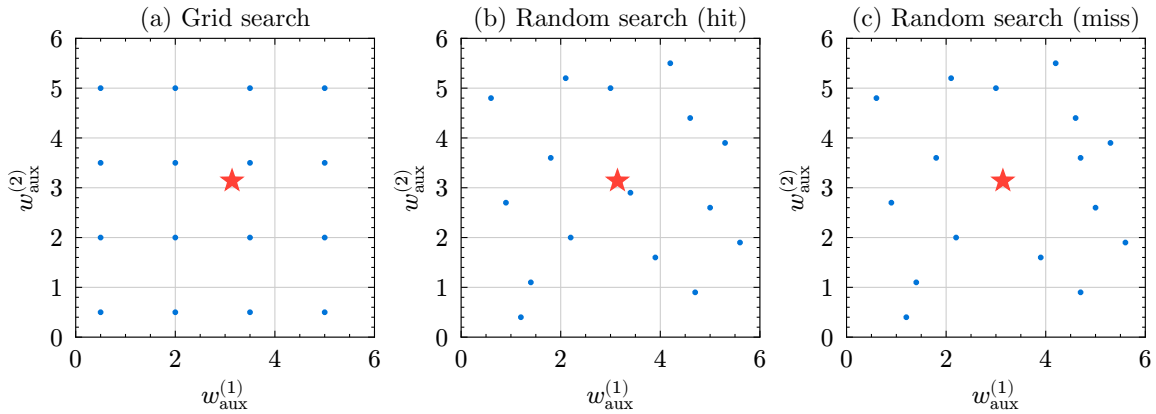


Figure 5.5: Grid and random search over two auxiliary loss weights. The red star indicates the optimal weight configuration. (a) shows that a grid search may miss the optimum, but is guaranteed to find a point within $\frac{\text{grid resolution}}{2}$ of the optimum. (b) shows that random search can find a point very close to the optimum, but (c) shows that it can also miss it entirely.

So-called "smart" hyperparameter tuning methods, such as Bayesian optimisation [63] and the tree-structured Parzen estimator (TPE) [64], build a model of the relationship between the weights and the agent's performance. This model is then used to guide the search for optimal weights, allowing for more efficient exploration of the search space. These methods can be more effective than grid or random search, but they can also be more complex to implement and still require multiple training runs. Methods such as Hyperband [65] and its asynchronous variant ASHA [66] mitigate this cost by giving each trial a small budget and only continuing the best-performing ones. However, such early-stopping methods may prematurely terminate RL trials that perform more exploration early on, despite performing better in the long run, so they are not ideal for our scenario.

We choose to use a simple grid search to find the optimal weights so that we can more rigidly study the effect of each auxiliary loss on the agent's performance. We modify the grid so that only one kind of auxiliary loss is active at a time so we can isolate the effect of each loss, and use weights $w_{\text{aux}} \in \{0.1, 0.3, 1.0\}$ - an exponential scale.

5.4.2. Adaptive Weighting

Adaptive weighting is an alternative approach that dynamically adjusts the weights of the auxiliary losses during training based on factors like the magnitude of the gradients or the error of the auxiliary predictions. Various methods have been introduced in the literature, such as GradNorm [67], which weights tasks that are learning slower more heavily, and Uncertainty Weighting [68], which uses the model's uncertainty in each task to determine the weights. These methods are useful in multi-task learning scenarios, where the performance of all tasks is important, but they are not ideal for our scenario, where the main task (driving) is the only one that matters. In this case, we want to combine auxiliary losses in such a way that improves the main task's performance, not to balance the performance of all tasks.

5.4.2.1. PCGrad

PCGrad [69] is a method that modifies the gradients of auxiliary tasks to prevent them from conflicting with the main task's gradients. It does so by projecting the auxiliary task gradients onto the normal plane of the main task's gradient if they conflict. Let $\boldsymbol{g}_{\text{aux}}^{(k)}$ be the gradient of the model parameters for the k-th auxiliary loss and $\boldsymbol{g}_{\text{PPO}}$ be the gradient of the main task loss. If the dot product $\boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{main}} < 0$, then the auxiliary gradient is projected onto the normal plane of the main task gradient:

$$\boldsymbol{g} = \boldsymbol{g}_{\text{PPO}} + \sum_{k=1}^{K} \begin{cases} \boldsymbol{g}_{\text{aux}}^{(k)} - \frac{\boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{PPO}}}{\|\boldsymbol{g}_{\text{PPO}}\|^2} \boldsymbol{g}_{\text{PPO}} & \text{if } \boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{PPO}} < 0 \\ \boldsymbol{g}_{\text{aux}}^{(k)} & \text{otherwise} \end{cases}$$

The result is that the auxiliary gradients are modified to avoid conflicting with the main task, but this does not guarantee that the auxiliary gradients

5.5. Metrics

5.5.1. Progress

Let L be the total length of the track centreline. At any time t, let $d_t \in [0, L)$ be the agent's current arc length from the start of the centerline to the point closest to the agent, and let N_t be the total number of completed laps. We define the agent's *absolute progress* along the track as the total continuous distance travelled from the start:

$$\xi_t = d_t + N_t L$$

The *progress per step* is then simply the change in this absolute position between consecutive timesteps, defined as $\xi_t - \xi_{t-1}$. Because ξ_t tracks the cumulative distance across multiple laps, this raw difference naturally handles crossing the finish line without requiring any wrapping operations or modulo arithmetic.

Using this step-wise progress as a reward signal is approximately equivalent to a potential-based reward shaping function [29]. In this context, the absolute progress ξ_t serves as the potential function $\Phi(s_t)$ of the current state. Because the reward is defined as (nearly) the difference in potential, $\Phi(s_t) - \gamma\Phi(s_{t-1})$, it heavily incentivises continuous forward movement along the track while theoretically being close to preserving the optimal policy and preventing the agent from exploiting positive-reward cycles.

5.5.2. Crash rates

The *crash rate per episode* is the fraction of total episodes that terminate in a collision rather than reaching the maximum length.

The *crash rate per timestep* is the total number of collisions divided by the total number of timesteps executed across all episodes. This can be interpreted as an estimate of the probability of a collision occurring at any given timestep, and is useful for evaluating the agent's safety in a time-normalised manner.

The *collisions per metre of progress along the centerline* is the total number of crashes divided by the total distance the agent successfully advances along the track centerline. This metric normalizes failures against spatial progression rather than time, isolating driving competence from the agent's velocity.

5.5.3. Reward function

The agent receives a dense reward at each timestep that encourages fast, collision-free progress along the track:

$$r_t = w_p \cdot (\xi_t - \xi_{t-1}) - w_c \cdot c_t$$

The first term rewards forward progress, using the progress per step defined above. The second term penalizes collisions: the indicator variable c_t equals 1 if a collision occurs with an obstacle at time t, and 0 otherwise.

The weights w_p (reward per metre of progress) and w_c (penalty per collision) balance speed against safety based on the risk tolerance of the application. We set $w_p = 1.0$ and $w_c = 1.0$. Because an episode terminates immediately upon collision, there is already a strong implicit penalty for crashing, allowing w_c to remain relatively low.

5.5.4. Area Under the Curve (AUC)

The *Area Under the Curve* (AUC) evaluates sample efficiency and learning stability by calculating the definite integral of a performance metric $y(t)$ (e.g., return or progress per step) over the total training time or timesteps T:

$$\text{AUC} = \int_0^T y(t)\,\mathrm{d}t$$

In practice, evaluation metrics are recorded at discrete interval checkpoints $t_0, t_1, ..., t_N$, and the AUC is computed using the trapezoidal rule:

$$\text{AUC} \approx \sum_{i=1}^{N} \frac{y(t_{i-1}) + y(t_i)}{2}(t_i - t_{i-1})$$

A higher AUC indicates faster learning and superior performance throughout the entire training process rather than merely achieving high final performance.

5.6. Trainer Performance Tuning

Training was performed on a single machine with an AMD Ryzen Threadripper PRO 3995WX CPU (64 cores, 128 hardware threads) and 128 GB of RAM. To ensure experiments finish in a practical time, we first measured how best to use this hardware, aiming to maximise *aggregate* throughput: environment timesteps collected and learned from per second, summed across all models training simultaneously.

We ran a grid search over the parameters in Table 5.2: concurrent trainers (2, 4, 8, 12), Python threads per trainer (8, 16) and parallel environments per trainer (4, 8, 10, 16). For each configuration, `concurrent_trainers` identical processes were launched at once, each running five PPO iterations (a 256-frame rollout plus four epochs of minibatch updates each iteration) with checkpointing and TensorBoard logging disabled. Configurations with more than 8 trainers and 16 environments (128 or more concurrent Unity instances) exhausted memory and were excluded, leaving 28 single-run configurations. Throughput per trainer was computed from the mean wall-clock time per iteration:

$$\text{throughput} = \frac{\text{rollout length} \times \text{parallel envs} \times \text{areas per env}}{\text{seconds per iteration}}$$

We report the throughput per trainer, showing how much each model slows as the machine gets busier, and summed across trainers, showing where adding trainers stops paying off.

As shown in Section Appendix J, total throughput peaks at 8 trainers (4172 timesteps/s with 8 environments and 16 threads) and falls at 12 (at most 3853 timesteps/s), while per-trainer throughput roughly halves from 2 to 8 trainers. The best settings use $8 \times 16 = 128$ python threads in total, matching the hardware thread count; at 12 trainers the 16-thread configurations are oversubscribed and fall behind the 8-thread ones. We therefore chose (`concurrent_trainers`, `num_inst`, `python_threads`) = (8, 8, 16), though its margin over the next-best configurations (4020 and 3962 timesteps/s) is within likely run-to-run variation. Using `num_areas` $\in \{8, 12\}$ per environment in this configuration did not help (3844 and 3999 timesteps/s).

By default, Unity spawns `n_cpus` garbage-collector threads; we set this to 8 with the `-gc-helper-count` launch argument for all runs.

Table 5.2: Parameters varied in the training performance grid search.

Parameter	Values considered	Description
`concurrent_trainers`	$\{2, 4, 8, 12\}$	Number of independent models training simultaneously, each in its own process.
`python_threads`	$\{8, 16\}$	Threads available to each trainer's PyTorch, OpenMP, and OpenBLAS thread pools.
`num_inst`	$\{4, 8, 10, 16\}$	Number of Unity environment instances per trainer. Each contains 6 training areas, so each instance steps 6 agents in parallel.

Chapter 6

Experiments & results

Each configuration of (auxiliary targets, auxiliary loss weights), as well as the baseline model, was trained for 600 PPO iterations with 256 frames per iteration on each of 48 concurrent environments, for a total of 7.3 million timesteps ($\approx$ 82 hours of sim-time). Each configuration was trained on 6 different seeds, and various statistics were recorded for each iteration of each run of each configuration.

6.1. Baseline Model Performance

The baseline model trained with the PPO loss and no auxiliary targets performed well in its primary task. It achieved a mean crash rate per meter of progress of 0.003369 ± 0.002793 crashes/m, indicating approximately one crash every 297 meters. It achieved a mean progress along the centerline per second of 4.13 ± 0.49 m/s ≈ 15 km/h.

Before we consider the effect of auxiliary losses, we first establish what the baseline model's internal representations encode.

6.2. Learned features of Baseline Model

Consider the network in Figure 5.2. Once trained, each layer l maps an input $\boldsymbol{x}$ to an intermediate representation $\boldsymbol{h}^{(l)} = f_l(\boldsymbol{x}; \theta)$, which becomes the input to the following layer. We would like to know what information these intermediate representations encode, and in particular whether they encode privileged state variables (e.g. velocity, slip, upcoming curvature).

Alain and Bengio [70] introduced *linear probes* for this purpose; the trunk network's weights are frozen, and a separate linear classifier (the probe) is trained in a supervised manner on the representations $\boldsymbol{h}^{(l)}$ of each layer to predict the desired target. Since the probe cannot change the representation, its test accuracy measures how much class information is *linearly* available at that layer. Anand et al. [71] extended this idea to reinforcement learning, probing an encoder's representations for ground-truth state variables of Atari games (read from the console's RAM).

The choice of a *linear* probe is deliberate; a sufficiently expressive probe may compute the target from almost any input, including the raw network input (LiDAR in our case), so high probe accuracy would say more about the probe than about the

representation [72]. Restricting the probe to a linear map means that it can only succeed if the network has already done the work of encoding the target along some direction in the representation space.

Many of the privileged variables of interest are continuous rather than categorical, so we use a linear *regression* probe instead of a classifier, as done by McGrath et al. [73] to probe AlphaZero for chess concepts and by Gurnee and Tegmark [74] to probe language models for spatial and temporal encodings.

To establish what the baseline represents without auxiliary supervision, linear probes were fit to the output of the final trunk layer to predict each auxiliary target at various points during the model's training. Each probe is an OLS regression model trained on activations from a ~35-minute rollout (52429 timesteps) and evaluated on a separate ~9-minute rollout (13107 timesteps). Figure 6.1 reports out-of-sample R^2 at initialisation and after 600 PPO iterations.

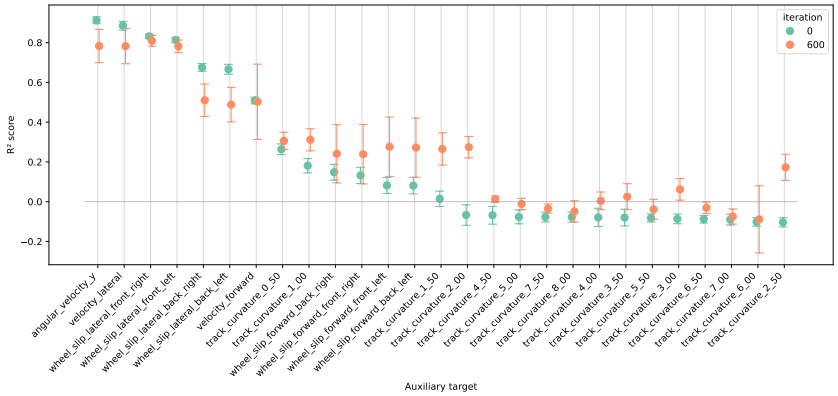


Figure 6.1: Out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the baseline model. Error bars show the standard deviation of the mean across 6 different seeds

Two distinct patterns emerge. Quantities describing the vehicle's instantaneous dynamics (yaw rate, lateral velocity and lateral wheel slip) are already linearly decodable at initialisation ($R^2 \approx 0.8$–0.9). Since the trunk network is untrained (with weights initialised using the LeCun method [75]), this reflects information that is close to linear in the observation and survives a series of random LeCun-initialised projections, rather than a learned representation. A similar effect was observed in [70] when investigating this phenomenon in toy MNIST models. Baseline PPO training does not improve the R^2 for these quantities, and decodability declines modestly (e.g. rear lateral slip falls from $R^2 = \sim 0.67$ to ~0.50), suggesting that PPO discards some of this information.

Figure 6.2 visualises the trend over time: as the model trains, the decodability of the instantaneous dynamics quantities declines, while the decodability of the close-ahead ($\leq 3m$) and medium-ahead ($3m < x \leq 6m$) track curvature improves almost monotonically (highlighted in boxes). This suggests that these quantities may prove to be useful auxiliary targets, as the auxiliary losses will encourage the model to learn their representations earlier. The model does not significantly learn to represent the far-

ahead curvature (6m - 8.0m), which presents itself as another candidate for an auxiliary target. If we are able to encourage the model to learn more about these quantities, it may be able to plan further ahead and improve its performance.

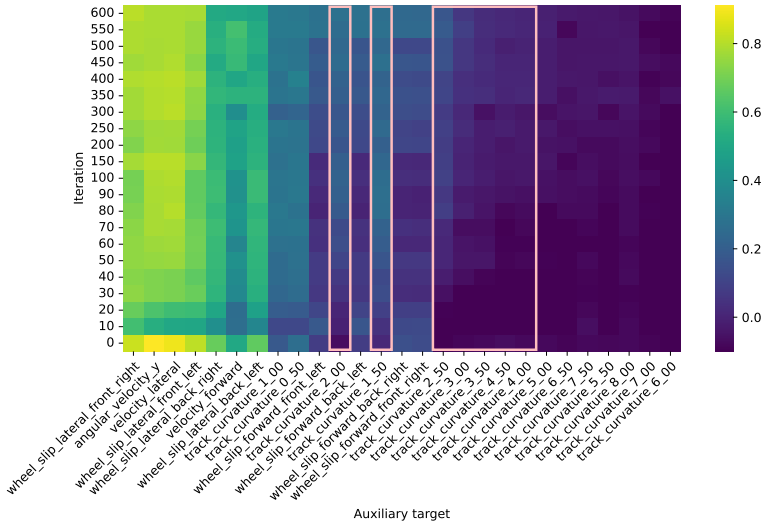


Figure 6.2: Mean out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the baseline model, showing how much each quantity is linearly decodable at each iteration of training. Note that the y-axis is not evenly spaced.

6.3. Agent Performance After Auxiliary Learning

The model is now trained with the same reward function as the baseline model (Section 5.5.3), but with the addition of one or more auxiliary heads (and their associated auxiliary losses).

The auxiliary head (and loss) that was most effective in improving the agent's performance was the close-ahead curvature head. It resulted in a measurably higher reward AUC compared to the baseline model, as visualised in Figure 6.3. Note that because the agent restarts after a collision, the reward AUC is only slightly influenced by the crash penalty (an average of 2.71 reward is generated per second vs -1 per crash). It is therefore important to monitor the performance of the agent in terms of each metric individually, as visualised in Figure 6.4, where the metrics (y-axis) are averaged over the last 50 training iterations and compared to the baseline model's mean. We find that the model trained with a loss for close-ahead curvature achieves both higher racing performance, as well as safety: it achieves a +13% mean progress per timestep along with a -38% collision rate per metre of progress at an auxiliary loss weight of $\omega_{\text{aux}} = 1.0$. Similar, though less pronounced results are achieved by the front wheel forward slip head, which is, in fact, safer (-59% collisions per metre of progress) than the baseline, and drives faster (+7% mean progress per timestep). Training the model with all auxiliary heads enabled, and with a small auxiliary loss weight of $\omega_{\text{aux}} = 0.1$, results in improved safety (-51% collisions per metre of progress) but no significant change in racing speed.

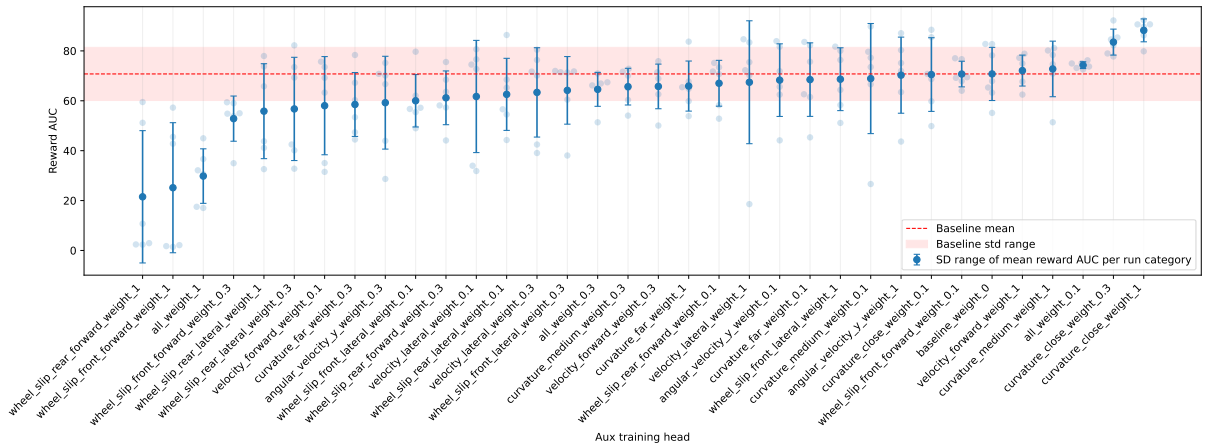


Figure 6.3: Reward AUC of the models trained with different auxiliary losses and loss weights over their 600 training iterations, compared to the baseline model

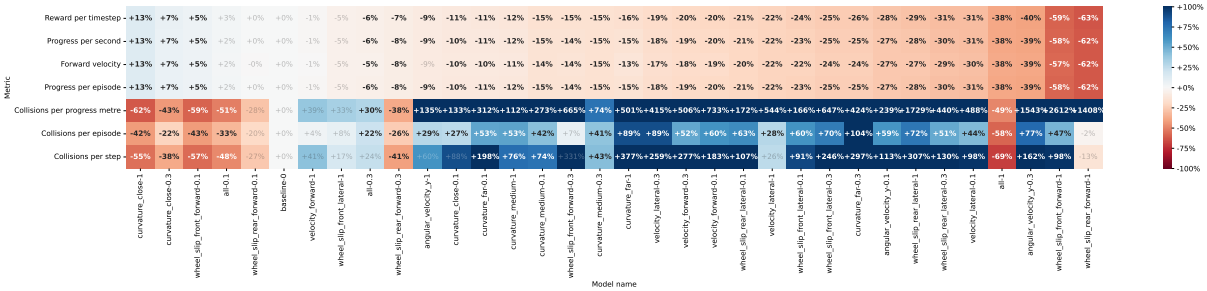


Figure 6.4: Mean of the last 50 training iterations of each model's metrics (mean reward, mean progress per timestep, mean forward velocity, mean number of collisions per metre of progress, mean number of collisions per episode, and mean number of collisions per timestep) vs the baseline model. Bold = statistically significant ($\alpha = 5\%$), using the Benjamini-Hochberg procedure for false discovery rate correction [76]

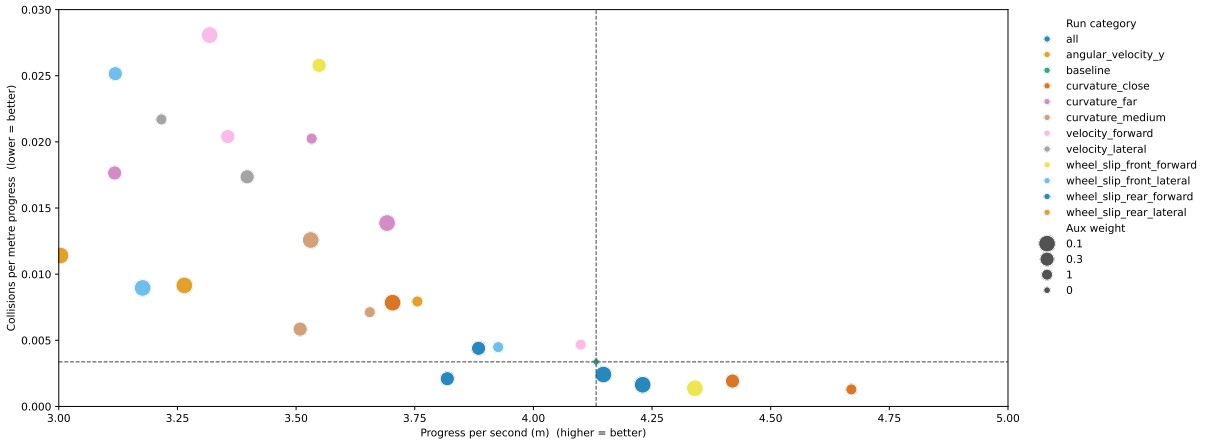


Figure 6.5: Comparison of different models' mean progress per second vs collisions per metre of progress. Bottom right quadrant = faster and safer than the baseline. Bottom left quadrant = slower but safer than the baseline.

6.4. Learned Features After Auxiliary Learning

Applying the same probing technique after auxiliary training, we find that decodability of the privileged quantities improved significantly in many configurations, though not always with a corresponding performance gain. The full results are visualised in Section Appendix I, since they are too numerous to include in the main text.

The close-ahead ($\leq 3m$) track curvature auxiliary loss was extremely effective in improving the decodability of the trunk's final layer. It improved the out-of-sample R^2 scores in close-ahead curvature quantities by between +16 to +58 percentage points, and even improved the decodability of quantities that were not directly supervised, such as 3.5-m-ahead curvature (up to +27 percentage points). The medum-ahead ($3m < x \leq 6m$) curvature auxiliary loss also improved the decodability of the trunk's final layer, but to a lesser extent.

When losses were introduced for the for wheel slip quantities, the decodability of the corresponding quantities improved significantly for smaller loss weigths ($w_{\text{aux}} \in \{0.1, 0.3\}$). Interestingly, when forward wheel slip was supervised for the front wheels, the decodability of the back wheel slip also increased. We hypothesis that this is because the car is driven with an all-wheel-drive configuration, so the front and back wheels are coupled.

Lateral slip supervision had mixed effects, with the decodability of the corresponding quantities improving significantly only for a larger loss weight ($w_{\text{aux}} = 1$).

When all auxiliary losses were enabled, the decodability of almost all quantities improved significantly, but the agent's performance only improved in terms of safety with a low auxiliary loss weight ($w_{\text{aux}} = 0.1$). This suggests that the auxiliary losses may be competing for the model's capacity when they are all enabled.

Chapter 7

Conclusions and recommendations

7.1. Summary

This project set out to determine whether auxiliary losses that predict privileged information improve the performance, training speed, and internal representations of a LiDAR-only end-to-end racing agent. All five objectives listed in Section 1.2 were met; a custom unity simulator was developed, a baseline PPO agent was trained (Section 5.3), linear regression probes established what the agent natively encodes (Figure 6.1). Then, auxiliary heads were added to predict privileged state information during training (Chapter 5), and the resulting agents were evaluated against the baseline model in terms of racing speed and safety, while also probing the internal representations of the trained agents (Section 6.4). The results show that auxiliary losses can improve both the agent's performance and its internal representations, but that this is not "free lunch": only certain auxiliary losses improve performance, and only at certain loss weights.

The baseline model already encoded its instantaneous dynamics well at initialisation. In contrast, the decodability of close-ahead and medium-ahead track curvature improved significantly during training with auxiliary heads, and the former had an impressive effect on the agent's driving capability, making it both safer and faster. Far-ahead curvature was not learned well by the model, and did not improve the agent's performance when supervised.

7.2. Future work

The results of this project suggest that more work is needed to determine *a priori* which auxiliary losses will improve the agent's performance, and at what loss weights, but that auxiliary losses can indeed improve both the agent's performance and its internal representations. Future work could explore the following avenues:

- Investigate the effect of auxiliary losses on the agent's performance in more complex environments, such as those with dynamic obstacles or varying track conditions.
- Explore the use of adaptive weighting methods for auxiliary losses, to dynamically adjust the weights during training based on the agent's performance and the difficulty of the auxiliary tasks.

- Investigate the effect of auxiliary losses on the agent's performance in multi-agent scenarios, where multiple agents are racing against each other

Bibliography

[1] World Health Organization, *Global Status Report on Road Safety 2018*. Geneva, Switzerland: World Health Organization, 2018, p. 403. Accessed: Oct. 22, 2026. [Online]. Available: https://www.who.int/publications/i/item/9789241565684

[2] J. Hankey *et al.*, "Identification and evaluation of driver errors: Task c report, driver error taxonomy development," *Project no. Dtfh-61-97-c-00051. Virginia Tech Transportation Institute, Blacksburg, VA*, 1999.

[3] A. L. Medina, S. E. Lee, W. W. Wierwille, and R. J. Hanowski, "Relationship between Infrastructure, Driver Error, and Critical Incidents," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 48, no. 16, pp. 2075–2079, 2004, doi: 10.1177/154193120404801661.

[4] B. Olofsson and L. Nielsen, "Using Crash Databases to Predict Effectiveness of New Autonomous Vehicle Maneuvers for Lane-Departure Injury Reduction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3479–3490, 2021, doi: 10.1109/TITS.2020.2983553.

[5] C. J. G. V. Driel and B. V. Arem, "The Impact of a Congestion Assistant on Traffic Flow Efficiency and Safety in Congested Traffic Caused by a Lane Drop," *Journal of Intelligent Transportation Systems*, vol. 14, no. 4, pp. 197–208, 2010, doi: 10.1080/15472450.2010.516226.

[6] H. Igliński and M. Babiak, "Analysis of the Potential of Autonomous Vehicles in Reducing the Emissions of Greenhouse Gases in Road Transport," *Procedia Engineering*, vol. 192, pp. 353–358, 2017, doi: https://doi.org/10.1016/j.proeng.2017.06.061.

[7] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable End-to-end Urban Autonomous Driving with Latent Deep Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2001.08726

[8] M. Jaderberg *et al.*, "Reinforcement Learning with Unsupervised Auxiliary Tasks," in *International Conference on Learning Representations (ICLR 2017)*, 2017. doi: 10.48550/arXiv.1611.05397.

[9] B. Rafiee, J. Jin, J. Luo, and A. White, "What makes useful auxiliary tasks in reinforcement learning: investigating the effect of the target policy." [Online]. Available: https://arxiv.org/abs/2204.00565

[10] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement

Learning," in *NeurIPS 2019 Competition and Demonstration Track*, 2020, pp. 77–89.

[11] B. D. Evans *et al.*, "Unifying F1TENTH Autonomous Racing: Survey, Methods and Benchmarks." [Online]. Available: https://arxiv.org/abs/2402.18558

[12] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A Survey of End-to-End Driving: Architectures and Training Methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1364–1384, Apr. 2022, doi: 10.1109/tnnls.2020.3043505.

[13] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020, doi: 10.1109/access.2020.2983149.

[14] P. R. Wurman *et al.*, "Outracing champion Gran Turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, Feb. 2022, doi: 10.1038/s41586-021-04357-7.

[15] H. Lee *et al.*, "A Champion-Level Vision-Based Reinforcement Learning Agent for Competitive Racing in Gran Turismo 7," *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 5545–5552, Jun. 2025, doi: 10.1109/lra.2025.3560873.

[16] G. Czechmanowski, J. Węgrzynowski, P. Kicki, and K. Walas, "On learning racing policies with reinforcement learning." [Online]. Available: https://arxiv.org/abs/2504.02420

[17] D. A. Pomerleau, "ALVINN: an autonomous land vehicle in a neural network," in *Proceedings of the 2nd International Conference on Neural Information Processing Systems*, in NIPS'88. Cambridge, MA, USA: MIT Press, 1988, pp. 305–313.

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[19] M. Bosello, R. Tse, and G. Pau, "Train in Austria, Race in Montecarlo: Generalized RL for Cross-Track F1tenth LIDAR-Based Races," in *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, 2022, pp. 290–298. doi: 10.1109/CCNC49033.2022.9700730.

[20] A. Srinivasan, "Reinforcement Learning: Advancements, Limitations, and Real-world Applications," *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, vol. 7, p. , Aug. 2023, doi: 10.55041/IJSREM25118.

[21] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models." [Online]. Available: https://arxiv.org/abs/1708.08296

[22] L. Chi and Y. Mu, "Deep Steering: Learning End-to-End Driving Model from Spatial and Temporal Visual Cues." [Online]. Available: https://arxiv.org/abs/1708.03798

[23] J. Mallon, “From Course to Competition: RoboRacer Students' Journey.” [Online]. Available: https://www.grasp.upenn.edu/news/grasp-graduate-students-roboracer/

[24] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, “Asymmetric Actor Critic for Image-Based Robot Learning.” [Online]. Available: https://arxiv.org/abs/1710.06542

[25] M. Vasco, T. Seno, K. Kawamoto, K. Subramanian, P. R. Wurman, and P. Stone, “A Super-human Vision-based Reinforcement Learning Agent for Autonomous Racing in Gran Turismo.” [Online]. Available: https://arxiv.org/abs/2406.12563

[26] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network.” [Online]. Available: https://arxiv.org/abs/1503.02531

[27] H. Xu, X. Chen, Y. Lang, and Q. Ren, “Vision-Based Policy Learning for High-Speed Autonomous Racing: A Two-Phase Learning Paradigm,” *IEEE Robotics & Automation Magazine*, vol. 33, no. 1, pp. 101–114, 2026, doi: 10.1109/MRA.2025.3639812.

[28] S. Ruder, “An Overview of Multi-Task Learning in Deep Neural Networks.” [Online]. Available: https://arxiv.org/abs/1706.05098

[29] A. Y. Ng, D. Harada, and S. J. Russell, “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,” in *Proceedings of the Sixteenth International Conference on Machine Learning*, in ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, pp. 278–287.

[30] A. Navon, I. Achituve, H. Maron, G. Chechik, and E. Fetaya, “Auxiliary Learning by Implicit Differentiation.” [Online]. Available: https://arxiv.org/abs/2007.02693

[31] G. Lample and D. S. Chaplot, “Playing FPS Games with Deep Reinforcement Learning,” in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI Press, 2017, pp. 2140–2146. [Online]. Available: https://arxiv.org/abs/1609.05521

[32] P. Mirowski *et al.*, “Learning to Navigate in Complex Environments.” [Online]. Available: https://arxiv.org/abs/1611.03673

[33] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, “TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving.” [Online]. Available: https://arxiv.org/abs/2205.15997

[34] Unity Technologies, *Unity ML-Agents Toolkit*. (2017). GitHub. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/Unity-Technologies/ml-agents

[35] A. Juliani *et al.*, “Unity: A General Platform for Intelligent Agents,” 2018. [Online]. Available: https://arxiv.org/abs/1809.02627

[36] T. V. Samak, C. V. Samak, and M. Xie, “AutoDRIVE Simulator: A Simulator for Scaled Autonomous Vehicle Research and Education,” in *2021 2nd International Conference on Control, Robotics and Intelligent System*, in CCRIS'21.

Qingdao, China: Association for Computing Machinery, 2021, pp. 1–5. doi: 10.1145/3483845.3483846.

[37] Unity Technologies, "Unity." [Online]. Available: https://unity.com/

[38] NVIDIA Corporation, "NVIDIA PhysX SDK." [Online]. Available: https://developer.nvidia.com/physx-sdk

[39] "Customised Usage Example." Accessed: Oct. 14, 2026. [Online]. Available: https://f1tenth-gym.readthedocs.io/en/v1.0.0/customized_usage.html#random-track-generator-beta

[40] C. V. Samak and T. V. Samak, "Autonomy Oriented Digital Twins for Real2Sim2Real Autoware Deployment." [Online]. Available: https://arxiv.org/abs/2402.14739

[41] "Unity Documentation." [Online]. Available: https://docs.unity3d.com/6000.5/Documentation/ScriptReference/WheelFrictionCurve.html

[42] H. B. Pacejka and E. Bakker, "THE MAGIC FORMULA TYRE MODEL," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, 1992, doi: 10.1080/00423119208969994.

[43] H. Pacejka, *Tyre and Vehicle Dynamics*. in Automotive Engineering Series. Elsevier Science & Technology Books, 2002. [Online]. Available: https://books.google.co.za/books?id=pQNUAAAAMAAJ

[44] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, "Quantifying Generalization in Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.

[45] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[46] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[47] A. Johnson, *Clipper2: Polygon Clipping, Offsetting & Triangulation in C++, C# and Delphi*. (2022). GitHub. Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/AngusJohnson/Clipper2

[48] B. R. Vatti, "A generic solution to polygon clipping," *Communications of the ACM*, vol. 35, no. 7, pp. 56–63, Jul. 1992, [Online]. Available: https://dl.acm.org/doi/10.1145/129902.129906

[49] G. W. Leibniz, "Meditatio nova de natura anguli contactus et osculi, horumque usu in practica Mathesi, ad figuras faciliore ac exactiore delineandas," *Acta Eruditorum*, pp. 289–292, Jun. 1686.

[50] D. J. Struik, Ed., *A Source Book in Mathematics, 1200-1800*. Cambridge, MA: Harvard University Press, 1969.

[51] J. Stewart, D. Clegg, and S. Watson, *Calculus: Early Transcendentals*. Cengage Learning, 2020. [Online]. Available: https://books.google.co.za/books?id=BCzLDwAAQBAJ

[52] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[53] S. P. Singh, T. Jaakkola, and M. I. Jordan, "Learning Without State-Estimation in Partially Observable Markovian Decision Processes," in *Machine Learning Proceedings 1994*, W. W. Cohen and H. Hirsh, Eds., San Francisco (CA): Morgan Kaufmann, 1994, pp. 284–292. doi: https://doi.org/10.1016/B978-1-55860-335-6.50042-8.

[54] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992, doi: 10.1007/BF00992696.

[55] P. Santana, "An Introduction to Deep Reinforcement and Imitation Learning." [Online]. Available: https://arxiv.org/abs/2512.08052

[56] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[57] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation." [Online]. Available: https://arxiv.org/abs/1506.02438

[58] "PyTorch: an imperative style, high-performance deep learning library," in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA: Curran Associates Inc., 2019.

[59] A. Bou *et al.*, "TorchRL: A data-driven decision-making library for PyTorch." [Online]. Available: https://arxiv.org/abs/2306.00577

[60] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.

[61] J. Terven, D. M. Cordova-Esparza, A. Ramirez-Pedraza, E. A. Chavez-Urbiola, and J. A. Romero-Gonzalez, "Loss Functions and Metrics in Deep Learning." [Online]. Available: https://arxiv.org/abs/2307.02694

[62] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[63] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *Advances in Neural Information Processing*

Systems, 2012, pp. 2951–2959. [Online]. Available: https://arxiv.org/abs/1206.2944

[64] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, 2011. [Online]. Available: https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

[65] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, pp. 1–52, 2018, [Online]. Available: https://arxiv.org/abs/1603.06560

[66] L. Li *et al.*, "A System for Massively Parallel Hyperparameter Tuning," in *Proceedings of Machine Learning and Systems*, 2020, pp. 230–246. [Online]. Available: https://arxiv.org/abs/1810.05934

[67] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," Nov. 2017. [Online]. Available: https://arxiv.org/abs/1711.02257

[68] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2018, pp. 7482–7491. [Online]. Available: https://arxiv.org/abs/1705.07115

[69] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, "Gradient Surgery for Multi-Task Learning." [Online]. Available: https://arxiv.org/abs/2001.06782

[70] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes." [Online]. Available: https://arxiv.org/abs/1610.01644

[71] A. Anand, E. Racah, S. Ozair, Y. Bengio, M.-A. Côté, and R. D. Hjelm, "Unsupervised State Representation Learning in Atari." [Online]. Available: https://arxiv.org/abs/1906.08226

[72] Y. Belinkov, "Probing Classifiers: Promises, Shortcomings, and Advances." [Online]. Available: https://arxiv.org/abs/2102.12452

[73] T. McGrath *et al.*, "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, p. e2206625119, 2022, doi: 10.1073/pnas.2206625119.

[74] W. Gurnee and M. Tegmark, "Language Models Represent Space and Time." [Online]. Available: https://arxiv.org/abs/2310.02207

[75] Y. LeCun, L. Bottou, G. B. Orr, and K. R. Müller, "Efficient BackProp," in *Neural Networks: Tricks of the Trade*, G. B. Orr and K.-R. Müller, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 9–50. doi: 10.1007/3-540-49430-8_2.

[76] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995, doi: https://doi.org/10.1111/j.2517-6161.1995.tb02031.x.

Appendix

Appendix C Project Timeline

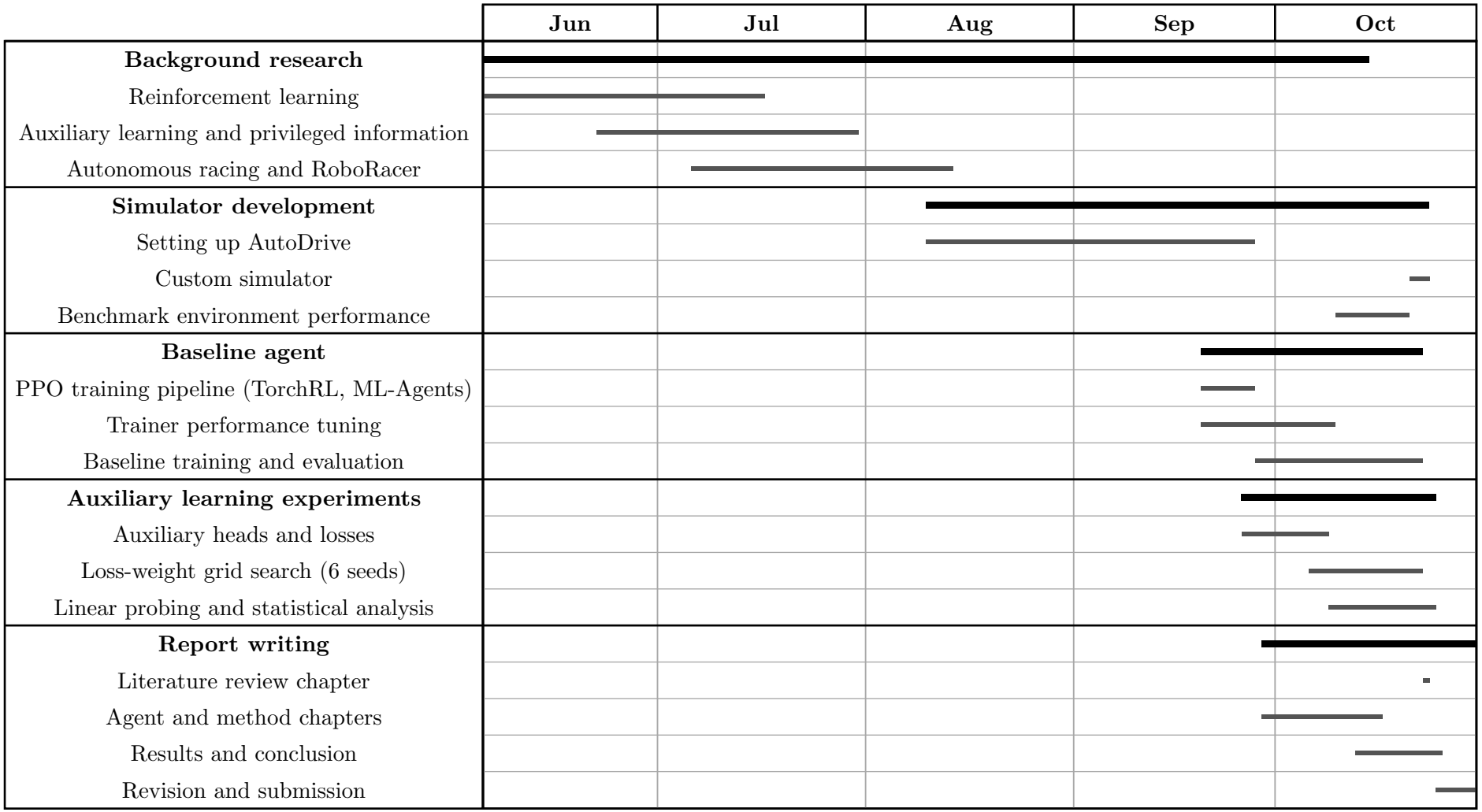


Figure 7.1: Timeline of the project, showing the main tasks and their durations.

Appendix D Graduate Attributes

Appendix E Complex Engineering Problem

The project addresses a complex engineering problem because it requires a fundementals-based formulation of autonomous racing as a POMDP and first-principles-based design of vehicle dynamic models. It draws on specialist knowledge of RL and autonomous racing, and engages with current research literature on privileged information, auxiliary learning, and representation probing. The problem has no obvious solution: a LiDAR-only agent must learn quantities it cannot observe, and it is not clear which privileged targets or loss weights would help. It involves conflicting requriements (speed vs. safety), balancing the loss weights of the primary task vs. auxiliary tasks, and considering the trade-offs between simulation fidelity and training speed. It also involves many coupled systems

Appendix F AI Declaration

- Claude was used for searching literature, checking the report for errors, advice on clarity of writing, and changing the format of references from BibTex to Hayagriva. Furthermore, Claude was used to explain various tensor shape bugs in PyTorch code, but the final implementation was always done manually, as well as to explain how TorchRL works. Initially, Claude was used to generate diagrams, but I did not like how little control I had over the diagrams, so re-implemented them manually from scratch. Claude was used to generate parts of Figure 4.5 based on a handmade diagram. Claude was used to explain why certain training setups may perform badly on the hardware available. Claude was used to explain how to perform the FDR analysis on Section Appendix I, but the code was written by me.
- Google Gemini was used to a lesser extent for searching literature, and, before starting the project, for assistance in understanding RL in conjunction with the David Silver Videos. Gemini was instrumental in explaining potential-based reward shaping in an easy-to-understand way. Gemini was used to do spelling & grammar checking.
- Google Scholar Labs was used extensively in searching for literature.
- DeepSeek was used to search for literature, and to provide input on the structure and content of the report. For example, it advised which sections could be reduced in length without sacrificing report quality.

AI was not used to generate the training code or the model definition code - that was completely hand-written.

Appendix G WheelCollider properties

Table 7.1: Relevant parameters of the WheelCollider model. * = forward & sideways

Property	Description	Value for RoboRacer
`radius`	Radius of the wheel	5.8 cm
`mass`	Mass of each wheel; determines its rotational inertia	109 g
`suspension_distance`	Maximum travel of the suspension along the wheel's local Y axis	5 cm
`suspension_spring`	Spring stiffness pulling the wheel toward its target position	500 N/m
`suspension_damper`	Damping that resists the speed of suspension movement	100 (N · s/m)
`extremum_slip`*	Slip at which the friction curve peaks (maximum grip)	0.15 fwd / 0.01 side
`extremum_value`*	Friction at the peak of the curve	0.9 fwd / 1 side
`asymptote_slip`*	Slip beyond which the tyre is fully sliding	0.25 fwd / 0.1 side
`asymptote_value`*	Friction while sliding	0.58 fwd / 0.5 side
`stiffness`*	Multiplier applied to the whole friction curve	0.8 fwd / 1 side

Appendix H Proportion of Data Within 1 Standard Deviation

Table 7.2: Proportion of the normalised auxiliary target data that lies within 1 standard deviation of the mean in the baseline rollout.

Auxiliary target	Prop. of data within 1 SD of mean
velocity_forward	90.85%
velocity_lateral	65.39%
track_curvature_0_50	76.73%
track_curvature_1_00	76.92%
track_curvature_1_50	75.08%
track_curvature_2_00	76.08%
track_curvature_2_50	77.36%
track_curvature_3_00	76.03%
track_curvature_3_50	76.98%
track_curvature_4_00	75.32%
track_curvature_4_50	77.61%
track_curvature_5_00	76.23%
track_curvature_5_50	75.49%
track_curvature_6_00	80.73%
track_curvature_6_50	76.31%
track_curvature_7_00	79.18%
track_curvature_7_50	77.84%
track_curvature_8_00	78.46%
wheel_slip_forward_front_left	95.19%
wheel_slip_lateral_front_left	74.03%
wheel_slip_forward_back_left	94.82%
wheel_slip_lateral_back_left	72.92%
wheel_slip_forward_front_right	95.2%
wheel_slip_lateral_front_right	72.35%
wheel_slip_forward_back_right	94.88%
wheel_slip_lateral_back_right	72.79%
angular_velocity_y	63.87%

Appendix I Auxiliary Head Regression Results

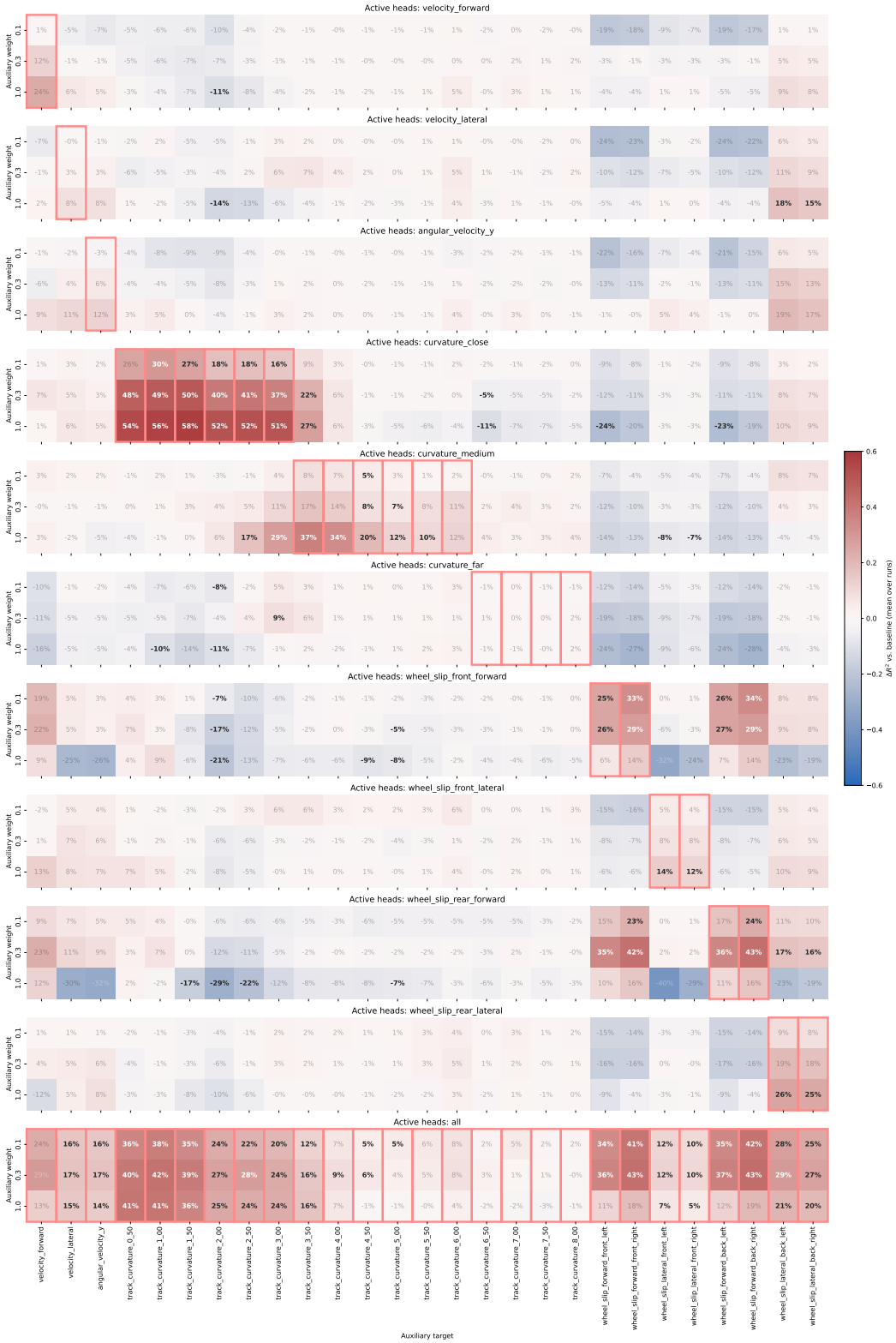


Figure 7.2: Difference in mean out-of-sample R^2 vs the baseline PPO model. The X-axis shows the auxiliary target, and the Y-axis shows the auxiliary training configuration. Red boxes indicate the features that are supervised by the auxiliary loss. Bold where the C.I. excludes 0, corrected for false discovery rate using the Benjamini-Hochberg procedure [76]

Appendix J Training Performance Benchmark Results

Table 7.3: Training performance results, sorted by throughput.

Concurrent trainers	Python threads	Parallel envs[5]	Throughput (timesteps/sec)	Speedup vs realtime[6]
8	16	8	4172.37	166.89x
8	16	4	4020.17	160.81x
8	16	8	3999.46	159.98x
8	16	10	3962.12	158.48x
12	8	4	3853.04	154.12x
8	16	8	3843.59	153.74x
12	16	10	3805.19	152.21x
8	8	4	3802.34	152.09x
12	8	8	3751.41	150.06x
12	8	10	3717.67	148.71x
12	16	8	3582.39	143.3x
4	16	10	3428.19	137.13x
12	16	4	3302.99	132.12x
8	8	8	3207.98	128.32x
4	16	8	3201.63	128.07x
8	8	10	3164.72	126.59x
4	16	4	2684.4	107.38x
4	8	4	2414.64	96.59x
2	16	10	2206.84	88.27x
4	8	16	2086.99	83.48x
4	16	16	2075.62	83.02x
2	16	8	2034.77	81.39x
4	8	10	1884.89	75.4x
4	8	8	1830.29	73.21x
2	16	4	1557.77	62.31x
2	8	4	1385.09	55.4x
2	8	16	1122.47	44.9x
2	16	16	1122.1	44.88x
2	8	10	1013.7	40.55x
2	8	8	977.93	39.12x

[5]It was not possible to run 16 parallel envs on 12 trainers, because the machine ran out of memory.
[6]Assuming 25 Hz decision frequency